

Instituto Politécnico Nacional

Escuela Superior de Cómputo

ESCOM

Trabajo Terminal I

Desarrollo de un General Game Player (GGP) utilizando un lenguaje de descripción de reglas para juegos de información perfecta

2027-A178

Ingeniería en Sistemas Computacionales

Presentan

Sánchez Catarino Edgar
Linares Villegas Gustavo

Director

Dr. en C. Enrique Escalante Notario



2026

INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
SUBDIRECCIÓN ACADÉMICA

No. de TT: 2027-A178 12 de mayo de 2026

Documento técnico

Desarrollo de un General Game Player (GGP) utilizando un lenguaje de descripción de reglas para juegos de información perfecta

2027-A178

Presentan

Sánchez Catarino Edgar

Linares Villegas Gustavo

Director

Dr. en C. Enrique Escalante Notario

RESUMEN

Este Trabajo Terminal propone diseñar e implementar un General Game Player (GGP) para juegos discretos, deterministas, por turnos y de información perfecta. La idea es separar las reglas de cada juego del motor de decisión del agente mediante un lenguaje de descripción externo. A partir de ese lenguaje, el sistema interpretará la estructura del juego, construirá una representación interna, generará los movimientos legales, aplicará transiciones, reconocerá los estados terminales y elegirá acciones con algoritmos generales de búsqueda. El proyecto se validará con el Gato, como caso funcional completo, y con escenarios controlados de Ajedrez, para observar qué tanto generaliza el enfoque sin modificar el núcleo algorítmico del agente.

Palabras clave: General Game Playing, lenguaje de descripción de juegos, inteligencia artificial, juegos de información perfecta, árboles de búsqueda, Monte Carlo Tree Search.

Advertencia

"Este documento contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional, a partir de datos y documentos sujetos a derechos de propiedad. Por lo tanto, su uso queda restringido a las aplicaciones que explícitamente se convengan."

Cualquier aplicación no convenida exime a la Escuela Superior de Cómputo de responsabilidad técnica y podrá dar lugar a las consecuencias legales que, para tal efecto, correspondan.

Para obtener información adicional sobre este reporte técnico, podrá dirigirse a:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional.

Índice general

Advertencia II

Glosario VIII

Abreviaturas XI

1 Introducción **1**
1.1 Contexto y motivación 1
1.2 Planteamiento del problema 2
1.3 Justificación 3
1.4 Objetivos 4
1.4.1 Objetivo general 4
1.4.2 Objetivos específicos 4
1.5 Alcance y limitaciones 4

2 Estado del arte **5**
2.1 Evolución de los sistemas de juego en inteligencia artificial 5
2.2 Surgimiento y fundamentos de General Game Playing 6
2.3 Generaciones y agentes emblemáticos de General Game Playing 6
2.3.1 Primera generación: Análisis estático de reglas y Minimax heurístico . . 6
2.3.2 Segunda generación: La revolución de Monte Carlo Tree Search 7
2.3.3 Tercera generación: PropNets optimizadas y heurísticas de simulación . 8
2.4 Plataformas y lenguajes modernos de descripción de juegos 8
2.4.1 Extensiones de GDL e información imperfecta 8
2.4.2 El sistema Ludii 10
2.4.3 Regular Boardgames 10
2.4.4 Entornos generales para aprendizaje: OpenSpiel 10
2.5 Tendencias contemporáneas en aprendizaje profundo y modelos unificados . . . 10
2.6 Síntesis comparativa y posicionamiento del Trabajo Terminal 11

3 Marco Teórico **14**
3.1 Inteligencia artificial en juegos 14
3.2 Juegos de información perfecta 14
3.2.1 Espacio de observaciones y función de observación (O_i) 15
3.2.2 Relación de indistinguibilidad ($\sim_i$) 15
3.2.3 La indistinguibilidad como relación de equivalencia 16

3.2.4 Conjuntos de información de Kuhn ($I_i(s)$) . . . 17
3.2.5 Ejemplos comparativos . . . 19
3.2.6 Implicación para el modelo formal Γ . . . 19
3.3 General Game Playing . . . 20
3.4 Modelo formal de juego . . . 21
3.4.1 Conjunto de roles (R) . . . 21
3.4.2 Espacio de estados (S) y semántica de fluents . . . 22
3.4.3 Estado inicial (s_0) . . . 23
3.4.4 Acciones individuales (A_i) y espacio conjunto de acciones ($\mathbf{A}$) . . . 23
3.4.5 Función de legalidad (L_i) . . . 24
3.4.6 Función de transición dinámica (T) . . . 24
3.4.7 Conjunto de estados terminales (F) . . . 24
3.4.8 Función de utilidad o meta (G_i) . . . 25
3.4.9 Axiomas de admisibilidad y juegos bien formados . . . 25
3.4.10 Distancia en el grafo de estados y estimación hacia configuraciones terminales . . . 25
3.4.11 Modelado síncrono de juegos por turnos alternados . . . 28
3.4.12 Interfaz computacional para el motor de decisión . . . 28
3.5 Lenguaje de descripción de juegos (GDL) . . . 28
3.5.1 Características fundamentales de GDL . . . 29
3.5.2 Concepto y estructura de un predicado lógico . . . 29
3.5.3 Predicados reservados del sistema en GDL . . . 30
3.5.4 Ejemplificación mediante código de especificación: El juego del Gato . . . 30
3.6 GDL, Ludii y lenguajes de dominio específico . . . 33
3.7 Intérprete y representación interna . . . 33
3.8 Árboles de juego y explosión combinatoria . . . 34
3.9 Minimax . . . 34
3.10 Poda alfa-beta . . . 35
3.11 Funciones heurísticas generales . . . 36
3.12 Monte Carlo Tree Search . . . 36
3.13 UCT . . . 36
3.14 Aprendizaje por refuerzo y aprendizaje profundo . . . 38
3.15 AlphaZero como antecedente moderno . . . 38
3.16 Síntesis del marco teórico . . . 38

4 Análisis **40**

5 Diseño **41**

6 Desarrollo **42**

7 Pruebas **43**

8 Conclusiones y trabajo a futuro **44**

Referencias **45**

Anexos **48**
8.1 Cronogramas 48

Índice de figuras

1.1 Comparación estructural entre un motor especializado tradicional y la arquitectura desacoplada de un General Game Player. 3

2.1 Comparación esquemática entre la inferencia lógica interpretada recursivamente y la evaluación acelerada por propagación de señales en redes proposicionales (*PropNets*). 7
2.2 Cronología evolutiva y generaciones algorítmicas en *General Game Playing* (GGP). 9

3.1 Representación en teoría de conjuntos de la función de observación $O_i : S \to \Omega_i$. En un entorno de información imperfecta (a), la función no es inyectiva y colapsa múltiples estados en una misma observación ($s_A \sim_2 s_B$), induciendo incertidumbre con $|I_2| > 1$. En un juego de información perfecta (b), la función es estrictamente inyectiva (mapeo uno a uno), garantizando que la preimagen de cada observación sea un conjunto unitario ($I(s) = \{s\}$) y eliminando toda incertidumbre sobre la configuración real del juego. 20
3.2 Árbol de juego adversarial con poda alfa-beta. Tras evaluarse el nodo con valor 2, el nodo MIN asegura un valor a lo sumo de 2, el cual es estrictamente inferior al valor $\alpha = 3$ ya garantizado por MAX en la rama izquierda; en consecuencia, se interrumpe la exploración del resto de los sucesores. 35
3.3 Ciclo iterativo de cuatro fases en Monte Carlo Tree Search (MCTS): Selección de nodos mediante la cota UCT, Expansión de nuevas acciones legales, Simulación estocástica (*rollout*) hasta un estado terminal y Retropropagación de recompensas hacia la raíz. 37
3.4 Flujo conceptual de ejecución y arquitectura desacoplada del General Game Player. 39

Índice de tablas

2.1 Comparación de paradigmas y sistemas representativos en juegos e inteligencia artificial. 12

3.1 Predicados reservados de GDL y componente de Γ que describen. 30

Glosario

Término	Definición
Acción legal	Movimiento que un jugador puede hacer en un estado concreto del juego, según las reglas de la descripción formal del dominio.
Acción nula (*noop*)	Acción pasiva (*no-operation*) que ejecutan los jugadores que no tienen el turno. Sirve para representar la alternancia de turnos dentro de un modelo formal síncrono y con varios participantes.
Agente inteligente	Sistema que percibe un entorno, razona sobre sus posibles acciones y elige una de acuerdo con un objetivo, una recompensa o un criterio de utilidad.
Árbol de juego	Grafo acíclico con las secuencias posibles de estados y acciones de una partida. Cada nodo es una configuración y cada arista, una acción conjunta que lleva a un nuevo estado.
Asunción de mundo cerrado (*CWA*)	Principio de la programación lógica según el cual todo hecho o *fluent* que no pueda deducirse como verdadero en el estado actual se considera falso.
Conjunto de información de Kuhn	Conjunto de estados que un participante no puede distinguir entre sí; reúne todas las configuraciones del entorno compatibles con lo que observa.
Criterio de selección UCT	Regla de selección de Monte Carlo Tree Search basada en la cota superior de confianza UCB1. Busca equilibrar la explotación de las jugadas que han funcionado con la exploración de las ramas poco visitadas.
Descripción de juego	Especificación externa y declarativa que define los roles, el estado inicial, las acciones legales, las transiciones, los estados terminales y las recompensas, sin programar esas reglas dentro del motor del agente.
Distancia geodésica	Número mínimo de transiciones legales conjuntas necesarias para ir de una configuración origen a una configuración destino en el grafo de estados del juego.
Estado	Configuración completa del juego en un instante discreto. Formalmente, es el conjunto finito de *fluents* atómicos verdaderos bajo la asunción de mundo cerrado.
Estado terminal	Configuración en la que la partida termina y se asigna una recompensa numérica a cada participante.
Explosión combinatoria	Crecimiento exponencial del número de configuraciones y de nodos del árbol de búsqueda a medida que se profundiza, provocado por el factor de ramificación del dominio.

Continúa en la siguiente página...

Término	**Definición**
Factor de ramificación	Número promedio de acciones legales que salen de un nodo del árbol de búsqueda de un juego.
Fluent	Proposición lógica atómica cuyo valor de verdad cambia con el tiempo o con el estado. Representa hechos del entorno que pueden aparecer, mantenerse o desaparecer tras cada transición.
Función de observación	Función que asigna a cada estado del juego la observación que realmente recibe un jugador, es decir, la vista de la configuración real que le llega.
Función heurística	Función que estima qué tan bueno es un estado no terminal, o qué tan cerca está de la victoria, cuando la explosión combinatoria impide explorar el árbol hasta el final.
General Game Player	Agente autónomo que recibe en tiempo de ejecución la descripción formal de un juego que no conocía y lo juega bien, sin ajustes manuales en su código para ese dominio.
Información perfecta	Propiedad de los juegos en los que la función de observación de cada participante es inyectiva, de modo que sus conjuntos de información tienen un solo elemento y no hay incertidumbre sobre el estado.
Intérprete	Módulo de software que analiza el archivo de reglas externo, valida su estructura y construye en memoria la representación del juego.
Juego bien formado (*well-formed game*)	Juego formal que cumple a la vez con la jugabilidad (no hay bloqueos o *deadlocks*) y con la terminación garantizada en un número finito de jugadas.
Lenguaje de descripción de juegos	Notación formal y declarativa de dominio específico (como GDL o Ludii) con la que se codifican la dinámica, la legalidad y las metas de un juego, separadas del algoritmo de decisión.
Minimax	Algoritmo clásico de decisión para juegos de dos jugadores y suma constante: elige la acción que maximiza la utilidad mínima que puede asegurar frente a un adversario racional.
Motor de decisión	Núcleo del agente que explora las jugadas posibles y elige una acción mediante búsqueda en árboles, podas o simulación estocástica.
Negación estratificada	Propiedad de la semántica de Datalog que prohíbe la recursión cruzada con negación, lo que asegura que el modelo lógico de las reglas sea único, finito y decidible.
Poda alfa-beta	Optimización de Minimax que descarta subárboles completos cuya exploración no puede cambiar la decisión óptima en la raíz.
Recompensa	Valor numérico que recibe cada rol al llegar a una configuración terminal y que mide qué tan bien le fue en la partida.
Relación de indistinguibilidad	Relación de equivalencia sobre los estados que vincula dos configuraciones cuando un participante recibe exactamente la misma observación de ambas.
Relajación de efectos negativos (*delete relaxation*)	Técnica heurística que ignora las condiciones que borran o invalidan *fluents*, con lo que es posible calcular en tiempo polinomial cotas inferiores a los estados terminales.

Continúa en la siguiente página...

Término	**Definición**
Retropropagación (*backpropagation*)	Fase del ciclo de MCTS en la que la recompensa obtenida en una hoja o en una simulación se propaga hacia la raíz y actualiza las estadísticas de visitas y el valor medio.
Simulación estocástica (*rollout*)	Fase de MCTS en la que se juega una partida rápida, con movimientos elegidos al azar, desde un nodo expandido hasta una condición terminal.
Transición	Función que, a partir del estado actual y de un vector de acciones conjuntas, determina el único estado sucesor.

Abreviaturas

Abreviatura	Significado
AI	Artificial Intelligence (Inteligencia Artificial)
CWA	Closed World Assumption (Asunción de Mundo Cerrado)
DSL	Domain-Specific Language (Lenguaje de Dominio Específico)
GDL	Game Description Language (Lenguaje de Descripción de Juegos)
GGP	General Game Playing o General Game Player, según el contexto
IA	Inteligencia Artificial
MCTS	Monte Carlo Tree Search (Búsqueda de Árbol Monte Carlo)
RL	Reinforcement Learning (Aprendizaje por Refuerzo)
TT	Trabajo Terminal
UCT	Upper Confidence Bounds applied to Trees

Capítulo 1

Introducción

1.1. Contexto y motivación

Los juegos acompañan a la inteligencia artificial desde sus primeros años. Sirven como banco de pruebas porque tienen reglas explícitas, espacios de estados discretos y resultados que cualquiera puede verificar, algo que el mundo físico rara vez ofrece [1]. En la década de 1950, Arthur L. Samuel escribió un programa para jugar a las damas inglesas que combinaba búsqueda Minimax con una evaluación heurística y que mejoraba con la experiencia acumulada [2]. Con él se popularizó el término *machine learning*, aunque la idea ya la había explorado en teoría Alan Turing [3]; lo que hizo Samuel fue llevarla a la práctica.

Las décadas siguientes dejaron varios hitos frente a jugadores humanos. En 1997, Deep Blue, la supercomputadora de IBM, derrotó al campeón mundial Garry Kasparov [4]. Lo logró con hardware masivamente paralelo y una búsqueda alfa-beta que examinaba hasta doscientos millones de posiciones por segundo, guiada por una función de evaluación pensada y ajustada solo para el ajedrez. Después, el sistema Chinook resolvió el juego de damas [5] y, en años más recientes, AlphaGo [6] y AlphaZero [7] alcanzaron un nivel sobrehumano en ajedrez, Shogi y Go combinando redes neuronales profundas con aprendizaje por refuerzo.

Aun así, estos sistemas comparten una limitación: cada uno está hecho para un solo dominio. Deep Blue no se diseñó para lidiar con la geometría del Go, el programa de Samuel no sabe nada de las reglas del ajedrez y la arquitectura de AlphaGo está atada a su tablero original. No es un defecto de implementación, sino una consecuencia de cómo se concibieron. Ni siquiera AlphaZero, que usa un mismo algoritmo de aprendizaje para tres juegos, escapa del todo, porque las reglas y las transiciones de cada juego siguen escritas a mano en el código de su simulador. AlphaZero no puede leer la descripción de un juego nuevo y entenderla por sí solo; antes alguien tiene que sentarse a programar qué jugadas son legales en ese tablero.

Para atacar esta limitación, a partir de 2005 la Universidad de Stanford y la *Association for the Advancement of Artificial Intelligence* (AAAI) impulsaron la línea de investigación conocida como *General Game Playing* (GGP) [8], [9]. Aquí el objetivo ya no es construir el mejor programa para un juego concreto, sino un agente que reciba la descripción formal de un juego que nunca ha visto y lo juegue bien, sin apoyarse en algoritmos o heurísticas escritos para ese dominio [10]. El trabajo de fondo pasa así del programador al agente, que debe interpretar la lógica del juego, armar sus propias representaciones internas y decidir en tiempo real.

1.2. Planteamiento del problema

Los agentes de juego tradicionales dependen demasiado de las reglas del entorno para el que fueron escritos. Sus estructuras de datos se diseñan a la medida del tablero y sus funciones heurísticas incorporan conocimiento empírico del dominio, como el valor relativo de las piezas en el ajedrez o tablas de posiciones ganadoras calculadas de antemano.

En General Game Playing el agente tiene que manejar el espacio de estados $\mathcal{S}$ de cualquier juego que le toque, sin saber de antemano cómo funciona. Desde la teoría de juegos, un juego de información perfecta se modela con una tupla formal: los jugadores, los estados válidos, las acciones permitidas, la función de transición y las utilidades terminales. El problema es el tamaño. Las configuraciones posibles crecen de forma exponencial y a veces ni siquiera se pueden recorrer completas.

Un ejemplo ayuda a ver la diferencia entre todas las configuraciones que una representación puede armar y las que el juego realmente admite. Tomemos el problema de las 8 damas como un juego de un solo jugador en un tablero de 8×8. Si el agente representa una posición como un vector $v = (v_1, v_2, \ldots, v_8)$, donde cada $v_i \in \{0, 1, \ldots, 7\}$ es la fila ocupada en la columna i, el **espacio de configuraciones de la representación** ($\mathcal{S}_{\text{rep}}$) es el producto cartesiano $\{0, 1, \ldots, 7\}^8$, con $8^8 = 16\,777\,216$ elementos. El **espacio de estados válidos** ($\mathcal{S}$) es bastante menor: si las reglas piden a lo sumo una dama por fila y por columna, quedan solo las permutaciones de ocho elementos, es decir $8! = 40\,320$ estados. Y dentro de estos, los **estados solución** ($\mathcal{S}_{\text{solución}} \subset \mathcal{S}$) son aquellos en los que ninguna dama ataca a otra, ni en fila, ni en columna, ni en diagonal. Para el tablero de 8×8 hay exactamente 92, que se reducen a 12 soluciones fundamentales si se consideran equivalentes las rotaciones y reflexiones. Frente a una configuración cualquiera, el agente no trae nada de esto cableado. Debe decidir por sí mismo si pertenece a $\mathcal{S}$ y si es una solución, y solo cuenta con las reglas declarativas que se le entregan.

En juegos adversariales más complejos el problema crece mucho. Para el ajedrez se estima que el número de posiciones legales alcanzables está entre 10^{43} y 10^{46} [11], [12], y el árbol de juego completo ronda las 10^{120} partidas posibles, la cifra conocida como número de Shannon [11]. Sin funciones de evaluación hechas a mano ni bibliotecas de aperturas dentro del núcleo, el agente tiene que leer el archivo de reglas mientras corre, calcular las acciones legales de cada turno y estimar qué tan buena es cada jugada, todo con límites estrictos de tiempo y memoria.

El problema que atiende este Trabajo Terminal es, entonces, cómo construir una arquitectura de software desacoplada, basada en un lenguaje de descripción de reglas y en un intérprete eficiente, de modo que un motor de decisión que no conoce el dominio pueda jugar varios juegos de información perfecta: que reconozca estados válidos, aplique transiciones y elija jugadas competitivas, sin tocar su código fuente. Qué tan competitivas resultan esas jugadas se medirá enfrentando al agente con agentes de referencia y con motores de nivel configurable.

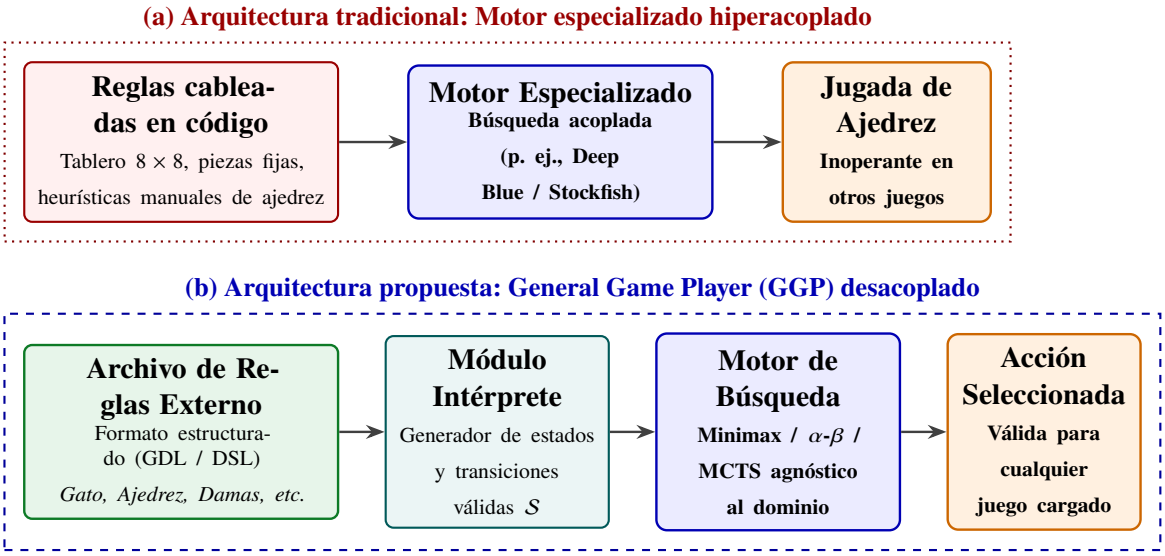


Figura 1.1: Comparación estructural entre un motor especializado tradicional y la arquitectura desacoplada de un General Game Player.

1.3. Justificación

El punto de partida de este trabajo es una incomodidad concreta: casi todos los agentes de juego quedan atados a un solo dominio. Tanto en la industria como en la academia lo habitual es dejar fijos, dentro del código fuente, las reglas, el tablero y las mecánicas. Un General Game Player que no dependa del dominio ofrece otra salida, porque el mismo programa se reutiliza con juegos distintos.

Lo distintivo de la propuesta es que el agente no sabe nada de ningún juego. No juega bien por consejos que dejó un experto humano, sino porque lee la descripción formal de las reglas y aplica procesos generales de búsqueda heurística y simulación. Las transiciones válidas y las condiciones de victoria salen del código ejecutable y pasan a un archivo externo estructurado; con eso se busca que el motor de inteligencia artificial quede separado por completo de la lógica de cada juego.

Desde la Ingeniería en Sistemas Computacionales, el desarrollo reúne varias áreas. Hace falta teoría de lenguajes formales y autómatas para diseñar el análisis léxico, sintáctico y semántico del intérprete de reglas; también modelado matemático y estructuras de datos eficientes para guardar en memoria espacios de estados enormes; y, por último, algoritmos de decisión adversarial, tanto clásicos como modernos, como Minimax con poda alfa-beta y Monte Carlo Tree Search (MCTS).

El proyecto, además, es viable en recursos y en tiempo dentro del ciclo anual de Trabajo Terminal en la Escuela Superior de Cómputo del Instituto Politécnico Nacional. No se pretende competir con los sistemas industriales de aprendizaje profundo, que requieren supercómputo, sino comprobar que una arquitectura modular, de escala académica, puede lograr generalidad procesando reglas declarativas y aplicando búsqueda heurística general.

1.4. Objetivos

1.4.1. Objetivo general

Diseñar e implementar un General Game Player (GGP) para juegos de información perfecta, capaz de interpretar las reglas de distintos entornos a partir de un lenguaje de descripción estructurado, basado en un modelo formal de juego. El agente debe ajustar su toma de decisiones a la especificación que se le cargue, sin modificar el código fuente de su motor algorítmico base.

1.4.2. Objetivos específicos

1. Diseñar la sintaxis y la semántica del lenguaje de descripción de reglas, es decir, el formato del archivo.

2. Desarrollar el intérprete y el generador de estados.

3. Implementar el motor de toma de decisiones del agente.

4. Validar la generalización del agente y del formato de archivo con dominios de reglas muy distintos, en particular el Gato (*Tic-Tac-Toe*) y el Ajedrez, de modo que el cambio de comportamiento dependa únicamente del archivo de especificación.

5. Evaluar el desempeño táctico del agente en ajedrez mediante partidas contra un motor especializado con ELO configurable, y obtener métricas como tasa de victorias, longitud de las partidas y tiempo de respuesta por jugada.

1.5. Alcance y limitaciones

Este Trabajo Terminal se limita a juegos que cumplen cuatro condiciones a la vez: información perfecta (el estado completo de la partida y el historial de jugadas son conocidos por todos los participantes en todo momento), estados y acciones discretos, dinámica determinista, es decir, sin azar, y turnos que se alternan de forma secuencial. Con esta frontera el esfuerzo se concentra en el lenguaje de reglas, el intérprete y los algoritmos de búsqueda general, sin sumar la complejidad de los modelos probabilísticos ni de los conjuntos de información parcialmente observables.

Elegimos dos casos que se parecen poco entre sí. El Gato (*Tic-Tac-Toe*) es tan pequeño que puede revisarse entero, y por eso ahí es posible comprobar de forma analítica que el intérprete y las transiciones funcionan bien. El Ajedrez está en el otro extremo: con tantas jugadas posibles y una táctica tan profunda, muestra qué le pasa al motor cuando el problema es grande.

También hay límites claros. El sistema no busca ganarle por fuerza táctica a motores comerciales muy especializados como Stockfish, que acumulan décadas de optimización específica en ensamblador, tablas de finales y grandes bases de aperturas [4], [13]. Tampoco se propone replicar sistemas de aprendizaje por refuerzo profundo como AlphaZero o Student of Games, que necesitan clústeres de alto rendimiento y entrenamientos largos [7], [14]. Lo que interesa es la generalidad: mostrar que un mismo núcleo de software puede cargar, interpretar y jugar partidas coherentes y competitivas en distintos juegos con solo cambiar el archivo de reglas, en una estación de trabajo convencional.

Capítulo 2

Estado del arte

2.1. Evolución de los sistemas de juego en inteligencia artificial

En ciencias de la computación, el estudio de los juegos ha seguido dos caminos. El primero, y el más antiguo, es construir motores muy especializados que rindan al máximo en un solo dominio. El segundo, que llegó después, busca agentes generales que entiendan y jueguen distintos juegos a partir de descripciones abstractas [1].

Al principio de esta historia, en 1959, Arthur Samuel mostró que una computadora podía mejorar en las damas inglesas mediante aprendizaje inductivo y una evaluación heurística de posiciones apoyada en Minimax [2]. Su trabajo abrió una larga tradición en la que cada juego se atacaba por separado, afinando a mano las funciones de evaluación y explorando árboles de juego de forma masiva.

Ese enfoque alcanzó su cima en 1997, cuando Deep Blue venció al campeón mundial Garry Kasparov [4]. Combinaba chips hechos a la medida para generar jugadas de ajedrez con una búsqueda alfa-beta muy paralelizada, que revisaba hasta doscientos millones de posiciones por segundo. Su criterio, sin embargo, lo habían calibrado a mano Grandes Maestros: pesos amarrados a las sesenta y cuatro casillas del tablero y al valor de cada pieza. Con Chinook pasó algo parecido en las damas, que resolvió en 2007 apoyándose en bases de datos exhaustivas de finales [5]. Y TD-Gammon llegó a nivel de maestro en backgammon con redes neuronales entrenadas por diferencias temporales.

Desde 2016 llegó otro salto, con AlphaGo [6], AlphaZero [7] y MuZero [15]. AlphaZero mostró que bastaban las reglas básicas para dominar el ajedrez, el Shogi y el Go. Lo hizo uniendo la búsqueda en árboles Monte Carlo (MCTS) con redes neuronales profundas entrenadas por autojuego, sin aperturas ni conocimiento humano experto.

Aun así, la literatura señala que AlphaZero no es un agente general en el sentido estricto [9], [16], [17]. El algoritmo de entrenamiento y búsqueda es el mismo para todos los juegos, pero las reglas de cada uno viven dentro de simuladores independientes escritos en C++. El sistema no puede leer la descripción de un juego nuevo ni deducir sus transiciones: primero un equipo de ingenieros debe programar el entorno. Esta limitación mostró que hacía falta una línea de investigación donde la generalidad del agente dependa de su capacidad para interpretar lenguajes declarativos formales.

2.2. Surgimiento y fundamentos de General Game Playing

Para atacar de raíz la hiperespecialización, en 2005 la Universidad de Stanford y la Asociación para el Avance de la Inteligencia Artificial (AAAI) lanzaron la iniciativa de *General Game Playing* (GGP) y convocaron a la primera competencia internacional abierta [8]. El documento fundacional, de Genesereth, Love y Pell, planteaba construir agentes autónomos que reciban la especificación formal de un juego cualquiera, desconocido de antemano, y lo jueguen bien sin supervisión humana ni cambios en su código.

Lo que hizo posible este esquema fue el *Game Description Language* (GDL) [9], [10]. El lenguaje se inspira en la programación lógica declarativa, en concreto en el subconjunto de Datalog con negación estratificada, y establece un compromiso claro: todo juego discreto, finito, síncrono y de información perfecta puede describirse como una máquina de estados abstracta usando unos pocos predicados lógicos.

Así transcurre una partida típica de GGP. El agente se conecta a un servidor central, que le manda el archivo de reglas unos minutos antes de empezar. Ese margen lo marca un reloj de preparación (*start clock*) y sirve para procesar la gramática, analizar el juego y armar las representaciones internas. Una vez que arranca la partida, cada turno tiene un reloj de jugada (*play clock*) para explorar el espacio de estados y responder. Con estas condiciones no hay lugar para heurísticas escritas de antemano: el agente tiene que derivar qué movimientos son legales, prever qué provocan y estimar qué tan cerca está del final, todo con la semántica de las reglas que recibió.

2.3. Generaciones y agentes emblemáticos de General Game Playing

En las competencias anuales de la AAAI, la literatura distingue tres generaciones de agentes [9], [18], [19]. Lo que las separa es cómo reparten el trabajo entre la deducción lógica y la exploración heurística.

2.3.1. Primera generación: Análisis estático de reglas y Minimax heurístico

En los primeros años, la idea fue llevar al caso general las técnicas clásicas de búsqueda adversarial, Minimax y poda alfa-beta. El problema es que, sin conocer el juego, el agente no tenía una función heurística con la cual evaluar los estados donde el árbol se corta por profundidad.

ClunePlayer, de Jim Clune en la Universidad de California en Los Ángeles, fue el primer campeón de GGP, en la competencia de la AAAI de 2005 [20]. Su aporte fue analizar la estructura sintáctica de las reglas GDL durante el tiempo de preparación para construir automáticamente una función de evaluación posicional. Identificaba heurísticas generales: la estabilidad de hechos (fluents que, una vez alcanzados, ya no pueden destruirse), la movilidad (cuántas jugadas legales tiene cada jugador) y la distancia simbólica a las condiciones de victoria definidas en las metas.

En 2006 el campeón fue Fluxplayer, de Stephan Schiffel y Michael Thielscher, en la Universidad Técnica de Dresde [21]. En lugar de inferencia lógica directa, usaba un razonador basado en el cálculo de fluents (*Fluent Calculus*) junto con aproximaciones de lógica difusa. Así podía medir cuán cerca estaba un estado cualquiera de cumplir las condiciones de victoria, y esas

evaluaciones continuas alimentaban una búsqueda alfa-beta con profundización iterativa. Ambos sistemas tenían, sin embargo, un mismo punto débil: cuando las reglas eran muy combinatorias o no lineales, analizar las cláusulas resultaba muy costoso y producía heurísticas engañosas que empeoraban las decisiones.

2.3.2. Segunda generación: La revolución de Monte Carlo Tree Search

El paso a la segunda generación cambió el campo por completo. Como era difícil inferir heurísticas estáticas confiables, los investigadores empezaron a probar métodos basados en simulación estocástica y en árboles de decisión que se adaptan sobre la marcha.

En 2007, Hilmar Finnsson y Yngvi Björnsson, de la Universidad de Reikiavik, presentaron CadiaPlayer [22]. CadiaPlayer dejó de lado el Minimax guiado por heurísticas sintácticas y aplicó por primera vez en GGP el algoritmo *Monte Carlo Tree Search* (MCTS) con la política de balance UCT (*Upper Confidence Bounds applied to Trees*) [19]. Lo interesante es que el agente no necesitaba entender a fondo las reglas para valorar una posición: simulaba cientos de partidas aleatorias desde el nodo actual hasta el final y promediaba las recompensas para decidir hacia dónde explorar. Ganó el campeonato mundial en 2007, 2008 y de nuevo en 2012, y desde entonces MCTS domina en General Game Playing.

En 2009 el primer lugar fue para Ary, de Jean Méhat en la Universidad de París 8, que implementó un motor MCTS muy paralelizado: corría simulaciones en hilos concurrentes y podaba de forma dinámica las ramas que resultaban estadísticamente desfavorables durante el muestreo.

En esa época apareció también una innovación clave en la representación interna de los estados: las redes proposicionales o PropNets. En vez de resolver consultas lógicas en cada nodo del árbol con un motor de Datalog, las reglas se compilaban en la fase de preparación como una red de compuertas booleanas (`AND`, `OR`, `NOT`) alojada directamente en memoria. Calcular el siguiente estado dejaba de ser un problema de resolución lógica y pasaba a ser la propagación de bits por un circuito, con lo que un motor MCTS podía correr varios órdenes de magnitud más simulaciones por segundo. La Figura 2.1 contrasta la deducción lógica interpretada con la aceleración por propagación de compuertas booleanas.

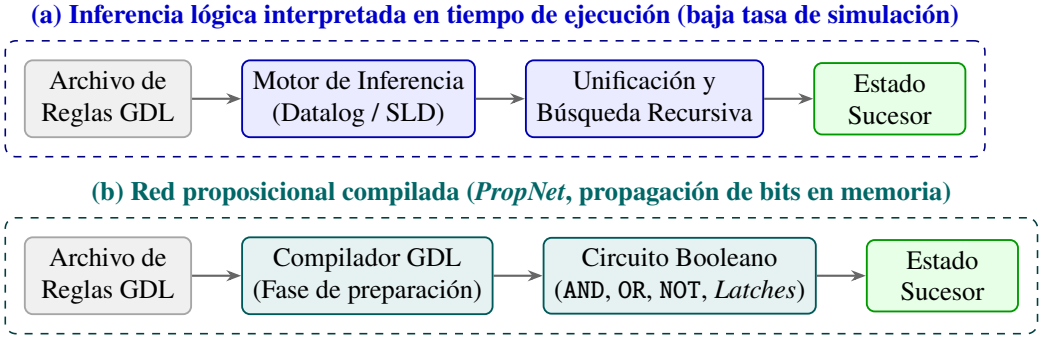


Figura 2.1: Comparación esquemática entre la inferencia lógica interpretada recursivamente y la evaluación acelerada por propagación de señales en redes proposicionales (*PropNets*).

2.3.3. Tercera generación: PropNets optimizadas y heurísticas de simulación

La tercera generación se dedicó a pulir el motor de simulación. El objetivo era superar la lentitud de las partidas puramente aleatorias en juegos con espacios de búsqueda inmensos o ramas terminales muy profundas.

Sancho, de Steve Draper y Andrew Rose, fue campeón internacional en 2014 [23]. Incorporaba una implementación industrial de PropNets muy optimizada y la combinaba con heurísticas de simulación informadas: MAST (*Move-Average Sampling Technique*), de Finnsson y Björnsson [22], y RAVE (*Rapid Action Value Estimation*), de Gelly y Silver [24]. Durante la simulación, en lugar de elegir los movimientos de manera uniforme, Sancho guardaba el valor promedio que había tenido cada acción en todas las simulaciones anteriores y sesgaba el muestreo hacia las jugadas que en conjunto habían resultado mejores.

Después llegó Galvanise, de Richard Emslie, que dominó los torneos internacionales desde 2015 con técnicas avanzadas de descomposición factorial del juego. Analizaba el grafo de dependencias de las reglas para ver si el juego contenía subentornos independientes, por ejemplo subpartidas en tableros paralelos, y aplicaba desramificación progresiva (*progressive unpruning*) junto con la detección de estados muertos. Así evitaba gastar tiempo de cómputo en ramas que, matemáticamente, no aportaban nada.

La Figura 2.2 resume cómo se pasa de una generación a otra y cómo todas desembocan en los formalismos actuales.

2.4. Plataformas y lenguajes modernos de descripción de juegos

GDL clásico llegó a ser el estándar académico para juegos discretos de información perfecta. Con su sintaxis prefija, heredada de KIF, y su semántica puramente declarativa, sin embargo, se volvía lento en tableros grandes o con mecánicas complejas. En la última década han aparecido plataformas y lenguajes alternativos que responden a esa limitación y han ampliado lo que se investiga.

2.4.1. Extensiones de GDL e información imperfecta

Para cubrir juegos con azar y con información oculta, Michael Thielscher formalizó en 2010 la extensión GDL-II (*Game Description Language for Incomplete Information and Stochastic Games*) [25]. Añadió al lenguaje los eventos estocásticos, mediante un rol especial llamado `random` que emite jugadas con distribución uniforme, como al tirar un dado, y la visibilidad parcial, mediante el predicado `sees`, que modela la función de observación $O_i : S \to \Omega_i$ para ocultar cartas o piezas al adversario. La expresividad creció bastante, pero también la complejidad del razonador, pues los agentes deben rastrear conjuntos de información de Kuhn y sostener creencias probabilísticas sobre los estados que no ven.

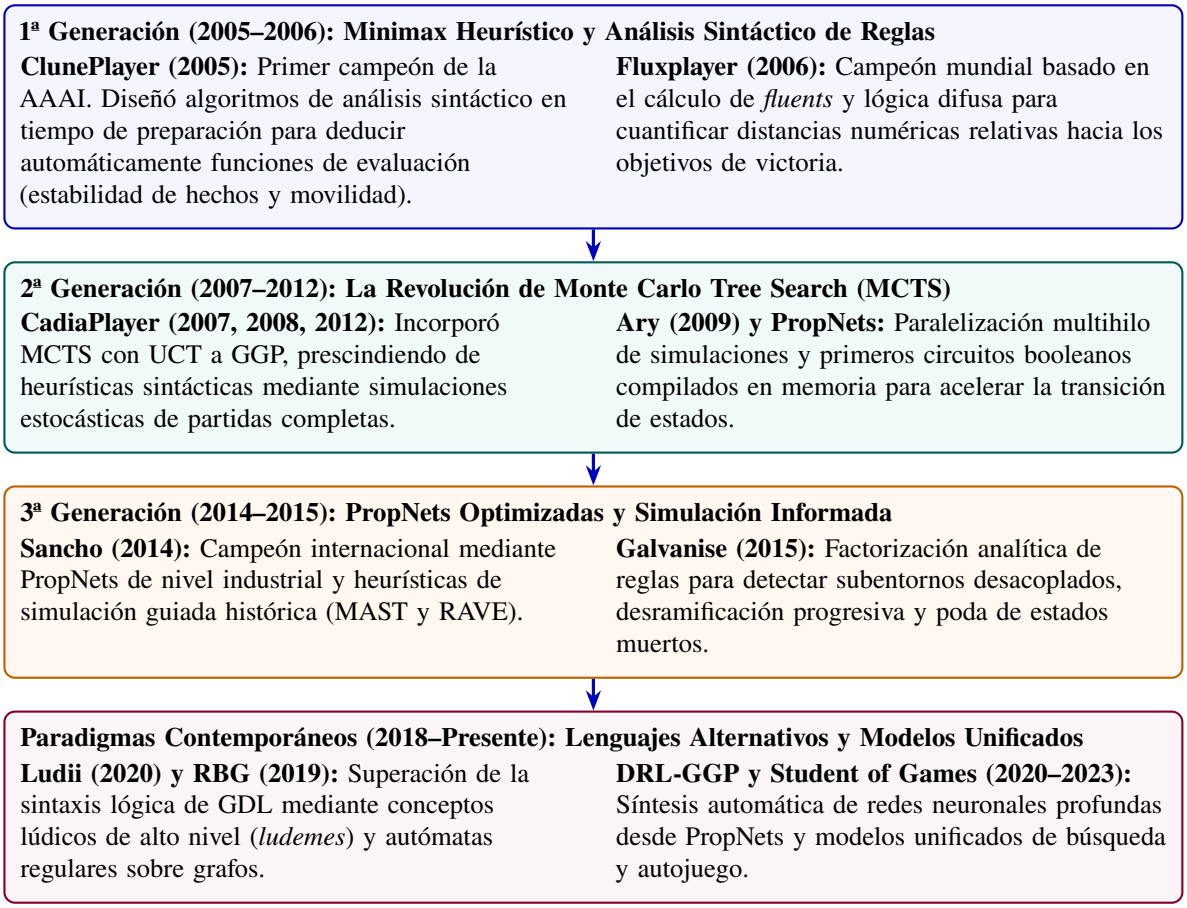


Figura 2.2: Cronología evolutiva y generaciones algorítmicas en *General Game Playing* (GGP).

2.4.2. El sistema Ludii

Ludii se desarrolló en la Universidad de Maastricht bajo la dirección de Cameron Browne y es uno de los avances más notables en la descripción general de juegos de mesa modernos [16], [26]. En lugar de la lógica de predicados pura de GDL, usa *ludemes*, unidades conceptuales que agrupan información lúdica de alto nivel.

El lenguaje trae abstracciones propias para tableros geométricos (cuadrículas, hexágonos, grafos arbitrarios), contenedores, piezas móviles, reglas de captura y condiciones de victoria por alineación o territorio. Como se ejecuta sobre una máquina virtual muy optimizada en Java, evalúa jugadas mucho más rápido que los intérpretes tradicionales de GDL, y su biblioteca ya supera los mil juegos tradicionales y contemporáneos [16], [26]. Para este trabajo importa porque demuestra que un lenguaje estructurado de más alto nivel puede seguir siendo general para la inteligencia artificial y, al mismo tiempo, ser más legible y eficiente.

2.4.3. Regular Boardgames

Regular Boardgames (RBG) es una propuesta de Jakub Kowalski, Maksymilian Mika, Jakub Sutowicz y Marek Szykuła, presentada en la conferencia AAAI de 2019. Se trata de un formalismo para juegos de tablero que se construye sobre lenguajes regulares y autómatas finitos [17].

En RBG, las reglas de movimiento y las condiciones de legalidad de cada pieza se escriben como expresiones regulares que describen trayectorias válidas sobre una cuadrícula vista como un grafo conexo. Los autores probaron que RBG es universal para juegos finitos, deterministas, por turnos y de información perfecta. Lo más llamativo es su rendimiento: los analizadores basados en autómatas generan movimientos legales y aplican transiciones entre dos y tres órdenes de magnitud más rápido que los razonadores lógicos de GDL, lo que lo hace muy adecuado para experimentos intensivos de búsqueda y de aprendizaje por refuerzo.

2.4.4. Entornos generales para aprendizaje: OpenSpiel

En el campo del aprendizaje automático actual, DeepMind liberó en 2019 la plataforma de código abierto OpenSpiel [27]. No es un lenguaje declarativo de reglas pensado para usuarios finales, sino un marco algorítmico en C++ y Python con una colección común de juegos e interfaces estándar para investigar aprendizaje por refuerzo, teoría de juegos algorítmica y búsqueda en árboles. Su valor está en ofrecer implementaciones de referencia de algoritmos modernos, como *Counterfactual Regret Minimization*, *Policy Gradients* y MCTS, con lo que se pueden comparar agentes en condiciones metodológicas homogéneas.

2.5. Tendencias contemporáneas en aprendizaje profundo y modelos unificados

Últimamente la investigación intenta juntar dos cosas: la descripción declarativa de juegos y las técnicas de aprendizaje profundo que transformaron a los motores especializados.

Un trabajo importante en esa dirección fue el de Adrian Goldwaser y Michael Thielscher, presentado en la conferencia AAAI de 2020, sobre aprendizaje por refuerzo profundo en GGP [28].

Mostraron que una descripción GDL puede compilarse en una red proposicional y que, a partir de la topología de compuertas de esa red, se puede generar de forma automática la arquitectura de una red neuronal convolucional o densa, sin que intervenga un diseñador humano. Después el agente entrena la red con autojuego continuo, en un esquema parecido al de AlphaZero, y consigue evaluaciones posicionales precisas en juegos de complejidad media.

Más general aún es *Student of Games* (SoG), que DeepMind presentó en 2023 y que Martin Schmid y colaboradores publicaron en *Science Advances* [14]. Es un algoritmo unificado que integra búsqueda guiada por árboles, aprendizaje por autojuego y teoría de juegos con resolución local continua. Compitió a nivel experto tanto en juegos de información perfecta (Ajedrez y Go) como en juegos de información imperfecta de gran escala (Póker Texas Hold'em y Scotland Yard), sin adaptar las ecuaciones de aprendizaje a cada caso. Con todo, igual que sus predecesores, depende de simuladores de reglas programados en software nativo y necesita semanas de entrenamiento en infraestructura de supercómputo [14]. Es un horizonte científico a largo plazo más que una solución de ingeniería ligera para un equipo de escritorio.

2.6. Síntesis comparativa y posicionamiento del Trabajo Terminal

Después de este recorrido se pueden comparar los enfoques principales de la literatura, como se resume en la Tabla 2.1.

Tabla 2.1: Comparación de paradigmas y sistemas representativos en juegos e inteligencia artificial.

Sistema / Paradigma	**Entrada de reglas**	**Motor de decisión**	**Tipo de juego**	**Dependencia de infraestructura**
Deep Blue [4]	Código nativo cableado	Alfa-beta especializado	Ajedrez (info. perfecta)	Hardware dedicado a la medida
AlphaZero [7]	Simulador nativo en C++	MCTS + Redes profundas	Varios (info. perfecta)	Clústeres masivos de TPUs
ClunePlayer [20]	Reglas lógicas (GDL)	Minimax con heurística aprendida	General GGP (info. perfecta)	Estación de trabajo estándar
CadiaPlayer [22]	Reglas lógicas (GDL)	MCTS con UCT sin heurística	General GGP (info. perfecta)	Estación de trabajo estándar
Sancho / Galvanise	Reglas lógicas (GDL)	MCTS + PropNets + MAST	General GGP (info. perfecta)	Estación de trabajo estándar
Ludii [16], [26]	Sintaxis de ludemes	MCTS / UCT / Alpha-beta	General (tableros y cartas)	Entorno Java de escritorio
RBG [17]	Expresiones regulares	Búsqueda con autómatas finitos	General (tableros)	Entorno C++ de alto rendimiento
Student of Games [14]	Simulador nativo en C++	Búsqueda + Teoría de juegos	Perfecta e imperfecta	Supercómputo distribuido
Presente TT (Propuesta)	**Lenguaje declarativo (DSL/GDL)**	**Minimax / Alfa-Beta / MCTS desacoplado**	**General (información perfecta)**	**Estación de trabajo estándar**

Del análisis se desprende dónde se ubica este Trabajo Terminal.

Primero, se aparta de las arquitecturas monolíticas de propósito único como Deep Blue y adopta el principio de General Game Playing: el motor de decisión no guarda variables ni heurísticas ligadas a ningún juego y razona únicamente sobre la máquina de estados que define el archivo de entrada.

Segundo, frente a propuestas de aprendizaje profundo como AlphaZero o Student of Games, que requieren clústeres enormes y simuladores hechos a mano para cada juego, este proyecto recupera el valor de un procesamiento declarativo, transparente y determinista. La generalidad del agente se apoya en interpretar reglas externas, y eso permite auditar las decisiones del motor, comprobar la legalidad de cada jugada y trabajar con los recursos de un equipo de escritorio.

Por último, el proyecto aprovecha lo aprendido en la evolución de GGP. Adopta una especificación declarativa estructurada, inspirada en el compromiso computacional de GDL, diseña un modelo interno en memoria pensado para evaluar transiciones con rapidez y ofrece una interfaz desacoplada con la que se pueden comparar los algoritmos clásicos de búsqueda adversarial con los métodos de simulación estocástica. Con ello se busca mostrar que sí es posible construir un agente de juego general, modular y eficiente dentro del marco de la Ingeniería en Sistemas

Computacionales.

Capítulo 3

Marco Teórico

3.1. Inteligencia artificial en juegos

Los juegos se han usado una y otra vez como banco de pruebas para la inteligencia artificial, porque tienen reglas explícitas, objetivos medibles y resultados que se pueden verificar. Desde los primeros trabajos de Arthur Samuel sobre aprendizaje automático en el juego de damas [2], han servido para estudiar búsqueda, planeación, toma de decisiones, aprendizaje y evaluación de estrategias.

Hay otra razón por la que los juegos importan: en un entorno controlado reúnen problemas que aparecen también fuera de ellos. Hay que elegir acciones bajo restricciones, anticipar consecuencias, competir con otros agentes, repartir el cómputo y decidir sin haber explorado todas las posibilidades. En un juego pequeño como el Gato todavía se puede recorrer buena parte del espacio de estados; en el Ajedrez, el número de estados y de secuencias de jugadas crece de forma combinatoria y sin búsqueda ni evaluación no hay manera de avanzar.

Históricamente, muchos de los sistemas más exitosos han sido motores especializados. Rinden bien porque llevan dentro conocimiento del dominio: reglas programadas para un juego, funciones de evaluación ajustadas a sus piezas o posiciones, representaciones hechas a la medida y optimizaciones para un tablero concreto. Eso da agentes fuertes, pero rígidos. Un programa hecho para jugar Ajedrez no puede, sin más, jugar Damas, Go o Gato, y no por falta de capacidad de cálculo, sino porque lo que sabe está pegado al juego para el que se construyó.

De ahí nace el interés por agentes que jueguen más de un juego sin reprogramar el motor, y esa línea de investigación es *General Game Playing*. Una parte del trabajo intelectual se mueve del programador al agente: ya no se espera que alguien codifique cada regla a mano, sino que el agente sepa leer la descripción formal del juego y razonar sobre ella.

3.2. Juegos de información perfecta

En teoría de juegos y en inteligencia artificial [1], [29], [30], algo que separa a los juegos es cuánto ven los jugadores del entorno. Un juego es de información perfecta cuando todos conocen, en todo momento, el estado completo y exacto de la partida: sin piezas ocultas, cartas boca abajo, movimientos secretos ni reglas desconocidas. El Ajedrez, las Damas y el Gato son los ejemplos de siempre.

Para formalizar esta idea con el modelo de juego (Γ) de la Sección 3.4, consideremos un conjunto de jugadores $R = \{r_1, r_2, \ldots, r_n\}$ y un conjunto de estados posibles S, donde cada estado $s \in S$ es la configuración completa y real del juego en un instante dado.

3.2.1. Espacio de observaciones y función de observación (O_i)

Conviene distinguir entre lo que el entorno es realmente y lo que cada jugador alcanza a percibir de él. Llamamos **estado real** ($s \in S$) a la configuración completa y verdadera del juego, que incluye todo lo que existe en ese instante: la posición de cada pieza, las cartas que cada participante tiene en la mano (también las que los demás no ven), el tiempo que queda en el reloj y a quién le toca mover. La **observación** ($o \in \Omega_i$), por su parte, es la información, parcial o total, que el jugador r_i recibe de ese estado, es decir, lo que ve en su pantalla o sobre la mesa. El conjunto Ω_i reúne todas las observaciones posibles para ese jugador.

La relación entre la realidad del juego y lo que ve el participante se modela con la **función de observación** [1], [25]:

$$O_i : S \to \Omega_i \tag{3.1}$$

que toma el estado real $s \in S$ y devuelve la observación $O_i(s) \in \Omega_i$ que ve el jugador r_i. Se puede pensar en O_i como un filtro colocado desde el punto de vista del jugador: aunque el juego esté en la configuración completa s, solo pasa lo que las reglas le permiten ver. En un juego de cartas, por ejemplo, el filtro borra las cartas del adversario y deja las propias y las comunitarias.

3.2.2. Relación de indistinguibilidad ($\sim_i$)

A partir de la función de observación se puede establecer con precisión cuándo dos situaciones distintas del juego resultan indiscernibles para un participante [30].

Definición 3.1: Relación de indistinguibilidad

Dado un conjunto de estados S, un espacio de observaciones Ω_i y una función de observación $O_i : S \to \Omega_i$ [25], [29], [30], la **relación de indistinguibilidad** $\sim_i \subseteq S \times S$ para el jugador $r_i \in R$ se define como:

$$s \sim_i s' \iff O_i(s) = O_i(s') \tag{3.2}$$

Dos estados $s, s' \in S$ son indistinguibles para el jugador r_i si y solo si producen exactamente la misma proyección perceptual en su codominio de observaciones.

Que dos estados s y s' sean indistinguibles para un jugador quiere decir que lo que los diferencia queda fuera de su campo de visión. Piénsese en un juego de cartas: en el estado s el adversario tiene un As oculto y en s' tiene un Rey oculto. El adversario los distingue sin problema, porque ve su propia mano. Para el jugador r_i, en cambio, la carta del rival está boca abajo y recibe exactamente la misma vista en ambos casos, $O_i(s) = O_i(s')$, de modo que no puede saber en cuál de las dos situaciones se encuentra.

3.2.3. La indistinguibilidad como relación de equivalencia

Para agrupar los estados que un jugador confunde entre sí hay que demostrar que $\sim_i$ es una relación de equivalencia sobre el espacio de estados [30].

Proposición 3.1. *Para todo jugador $r_i \in R$, la relación binaria de indistinguibilidad $\sim_i$ es una relación de equivalencia sobre el conjunto de estados S.*

Demostración (véase [30]). Se debe verificar que $\sim_i$ satisface las tres propiedades fundamentales:

1. **Reflexividad ($\forall s \in S : s \sim_i s$):** Sea $s \in S$ un estado arbitrario. La observación que produce dicho estado es idéntica a sí misma:

   $$O_i(s) = O_i(s)$$

   Por tanto, $s \sim_i s$ para todo $s \in S$.

   Un estado siempre produce la misma vista que él mismo, así que ningún jugador puede confundirlo consigo.

2. **Simetría ($\forall s, s' \in S : s \sim_i s' \implies s' \sim_i s$):** Sean $s, s' \in S$ tales que $s \sim_i s'$. Por definición, ambos estados producen la misma observación: $O_i(s) = O_i(s')$. En virtud de la simetría de la igualdad matemática, se tiene:

   $$O_i(s') = O_i(s)$$

   lo que implica que $s' \sim_i s$.

   La confusión es recíproca: si la vista de s no permite descartar que el juego esté en s', la vista de s' tampoco permite descartar que esté en s.

3. **Transitividad ($\forall s, s', s'' \in S : (s \sim_i s' \land s' \sim_i s'') \implies s \sim_i s''$):** Sean $s, s', s'' \in S$ tales que $s \sim_i s'$ y $s' \sim_i s''$. Esto significa que:

   $$O_i(s) = O_i(s') \quad \text{y} \quad O_i(s') = O_i(s'')$$

   Por la propiedad transitiva de la igualdad matemática:

   $$O_i(s) = O_i(s'')$$

   lo cual demuestra que $s \sim_i s''$.

   Si s se ve igual que s' y s' se ve igual que s'', los tres estados se ven idénticos para el jugador, y por tanto s y s'' son indistinguibles.

Al satisfacerse la reflexividad, simetría y transitividad, queda formalmente demostrado que $\sim_i$ es una relación de equivalencia sobre S. ■

3.2.4. Conjuntos de información de Kuhn ($I_i(s)$)

Toda relación de equivalencia clasifica los elementos de un conjunto en bloques disjuntos, llamados clases de equivalencia:

$$\mathcal{I}_i = S/\sim_i = \{[s]_{\sim_i} \mid s \in S\} \tag{3.3}$$

En la teoría matemática de juegos formulada originalmente por Kuhn [29] y desarrollada en textos canónicos de teoría de juegos [30], a cada clase de equivalencia $[s]_{\sim_i}$ se le llama **conjunto de información** del jugador r_i ante el estado s, denotado formalmente como $I_i(s)$.

Definición 3.2: Conjunto de información de Kuhn

Para un jugador $r_i \in R$ y un estado real del juego $s \in S$, el **conjunto de información** $I_i(s)$ se define formalmente, siguiendo la formulación canónica de juegos en forma extensiva de Kuhn [29], [30], como la clase de equivalencia inducida por la relación de indistinguibilidad $\sim_i$:

$$I_i(s) = [s]_{\sim_i} = \{s' \in S \mid s \sim_i s'\} = \{s' \in S \mid O_i(s') = O_i(s)\} \tag{3.4}$$

$I_i(s)$ reúne la totalidad de estados posibles del juego que resultan compatibles con la observación perceptible por el participante en dicho instante.

$I_i(s)$ expresa la duda del participante. Cuando el juego está en el estado real s, el jugador no puede afirmar con certeza que "el juego está en s"; lo único que puede sostener es que "el juego está en alguno de los estados de $I_i(s)$".

En los juegos de información imperfecta, la función de observación O_i **no es inyectiva**: existen al menos dos estados distintos que producen la misma observación ($\exists s, s' \in S, s \neq s' : O_i(s) = O_i(s')$). La preimagen de esa observación contiene entonces varios estados y forma un conjunto de información no trivial ($|I_i(s)| > 1$). El participante enfrenta una incertidumbre real, porque no puede saber en cuál de ellos está y debe apoyarse en distribuciones de probabilidad o en creencias para decidir su movimiento.

En los juegos de información perfecta ocurre lo contrario: O_i es **estrictamente inyectiva**, y cada estado real produce una observación única que lo identifica sin ambigüedad:

$$\forall s, s' \in S: \quad O_i(s) = O_i(s') \implies s = s' \tag{3.5}$$

Al ser un mapeo uno a uno, ningún par de estados distintos coincide en su imagen, y la preimagen de cualquier observación es forzosamente un conjunto unitario (*singleton*):

$$O_i^{-1}(O_i(s)) = I_i(s) = \{s\} \tag{3.6}$$

No queda duda ni ambigüedad perceptual: para cada jugador, el abanico de posibilidades se reduce a la realidad misma.

Definición 3.3: Juego de información perfecta

Un juego formal con conjunto de jugadores R y conjunto de estados S es de **información perfecta** [25], [29], [30] si y solo si, para todo jugador $r_i \in R$ y para todo estado $s \in S$, su

conjunto de información contiene únicamente el estado real:

$$\forall r_i \in R, \quad \forall s \in S : \quad I_i(s) = \{s\}$$

En otras palabras, el conjunto de información de cada participante tiene siempre un solo elemento (*singleton*), y por eso nadie duda nunca de cuál es la configuración exacta del juego.

Proposición 3.2 (Equivalencia con la inyectividad de la función de observación). *Un juego formal con espacio de estados S y conjunto de participantes R es de información perfecta si y solo si, para todo jugador $r_i \in R$, su función de observación $O_i : S \to \Omega_i$ es estrictamente inyectiva.*

Demostración. Se demuestra la doble implicación ($\iff$):

1. **Necesidad ($\implies$):** Supóngase que el juego es de información perfecta. Por la Definición 3.3, para todo $s \in S$ y todo $r_i \in R$, el conjunto de información colapsa a un conjunto unitario:
$$I_i(s) = [s]_{\sim_i} = \{s' \in S \mid O_i(s') = O_i(s)\} = \{s\}$$
Sean $s_1, s_2 \in S$ dos estados arbitrarios tales que producen la misma observación: $O_i(s_1) = O_i(s_2)$. Por la definición de la clase de equivalencia, esto implica directamente que $s_1 \in I_i(s_2)$. Dado que el juego es de información perfecta, se tiene que $I_i(s_2) = \{s_2\}$, de lo cual se deduce de manera unívoca que:
$$s_1 = s_2$$
Por tanto, $O_i(s_1) = O_i(s_2) \implies s_1 = s_2$, demostrando que O_i satisface formalmente la definición de una función inyectiva.

2. **Suficiencia ($\impliedby$):** Supóngase que la función de observación $O_i : S \to \Omega_i$ es estrictamente inyectiva. Sea $s \in S$ un estado arbitrario y considérese su respectivo conjunto de información:
$$I_i(s) = \{s' \in S \mid O_i(s') = O_i(s)\}$$
Por reflexividad de la igualdad, siempre se cumple que $s \in I_i(s)$. Sea ahora cualquier estado $s' \in I_i(s)$. Por definición de pertenencia, se tiene forzosamente que $O_i(s') = O_i(s)$. Puesto que la función O_i es inyectiva por hipótesis, la igualdad de imágenes exige estrictamente que:
$$s' = s$$
En consecuencia, no puede existir ningún elemento en $I_i(s)$ distinto de s, concluyéndose que:
$$I_i(s) = \{s\}$$
Al verificarse esta condición para todo jugador $r_i \in R$ y para todo estado $s \in S$, el juego es de información perfecta.

Queda formalmente demostrada la equivalencia lógica. ■

Esta equivalencia deja ver dos situaciones muy distintas para un agente inteligente. Si el juego es de información imperfecta (O_i no inyectiva), hay al menos dos estados $s \neq s'$ con la misma vista ($O_i(s) = O_i(s') = o$). Como el mapeo es de muchos a uno, la preimagen o fibra $O_i^{-1}(o) = I_i(s)$ agrupa varias configuraciones ($|I_i(s)| > 1$). Esa es la raíz matemática de la incertidumbre de Kuhn: el jugador recibe la observación o, pero no tiene manera determinista de invertir la función y decidir si el estado verdadero es s o s'.

Si, en cambio, el juego es de información perfecta (O_i estrictamente inyectiva), ningún par de estados distintos comparte imagen y la preimagen de cualquier observación es siempre un conjunto unitario: $O_i^{-1}(O_i(s)) = I_i(s) = \{s\}$. El participante puede entonces invertir la función de observación y reconstruir el estado exacto del mundo:

$$s = O_i^{-1}(o) \tag{3.7}$$

sin asimetrías perceptuales ni incertidumbre táctica sobre el entorno.

3.2.5. Ejemplos comparativos

Dos ejemplos ayudan a ver la diferencia.

En el Gato ($R = \{\mathrm{x}, \mathrm{o}\}$), que es un juego de información perfecta, el tablero de 3×3 está completamente a la vista de ambos jugadores. Si el estado real es s (por ejemplo, una ficha en el centro), la función de observación de cada participante devuelve exactamente ese tablero: $O_{\mathrm{x}}(s) = s$ y $O_{\mathrm{o}}(s) = s$. Sus conjuntos de información son, por tanto,

$$I_{\mathrm{x}}(s) = \{s\}, \quad I_{\mathrm{o}}(s) = \{s\} \tag{3.8}$$

Ninguno duda de qué casillas están marcadas; los dos ven toda la realidad del juego a través de un mapeo inyectivo directo. Lo mismo vale para el Ajedrez.

La Batalla Naval, en cambio, es de información imperfecta. Supongamos que el jugador 1 coloca su barco en la coordenada $(1, 1)$ (estado s_A) o en la $(1, 2)$ (estado s_B). Él sabe dónde puso su nave, así que distingue las dos situaciones: $I_1(s_A) = \{s_A\}$ e $I_1(s_B) = \{s_B\}$. El jugador 2, sin embargo, tiene la cuadrícula rival cubierta por niebla, y para él ambos tableros son idénticos: $O_2(s_A) = O_2(s_B) = o$. La función no es inyectiva ($s_A \sim_2 s_B$) y resulta

$$I_2(s_A) = I_2(s_B) = \{s_A, s_B\} \tag{3.9}$$

El conjunto de información del jugador 2 tiene dos estados ($|I_2| = 2 > 1$). No sabe si el barco enemigo está en $(1, 1)$ o en $(1, 2)$ y se ve obligado a adivinar o a estimar probabilidades.

La Figura 3.1 ilustra esquemáticamente esta distinción a través de la representación canónica en teoría de conjuntos de funciones inyectivas y no inyectivas.

3.2.6. Implicación para el modelo formal Γ

Que un juego cumpla $I_i(s) = \{s\}$ para todos los participantes simplifica bastante el modelo matemático Γ de la Sección 3.4. Por un lado, la función de legalidad $L_i(s)$ y la de utilidad $G_i(s)$ toman directamente como argumento el estado real $s \in S$, de modo que el motor no necesita estimar variables ocultas ni ponderar probabilidades de observación: el jugador siempre conoce

(a) Información imperfecta: función no inyectiva (b) Información perfecta: función inyectiva (uno a uno)

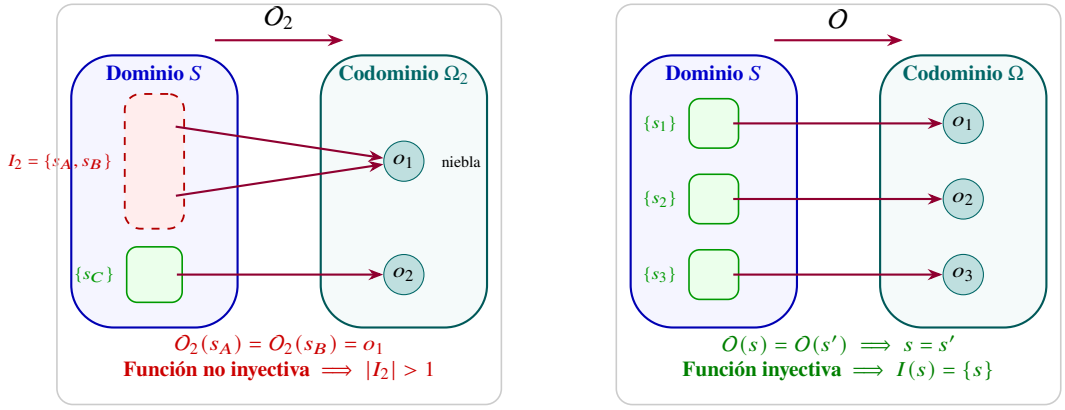


Figura 3.1: Representación en teoría de conjuntos de la función de observación $O_i : S \to \Omega_i$. En un entorno de información imperfecta (a), la función no es inyectiva y colapsa múltiples estados en una misma observación ($s_A \sim_2 s_B$), induciendo incertidumbre con $|I_2| > 1$. En un juego de información perfecta (b), la función es estrictamente inyectiva (mapeo uno a uno), garantizando que la preimagen de cada observación sea un conjunto unitario ($I(s) = \{s\}$) y eliminando toda incertidumbre sobre la configuración real del juego.

el estado exacto. Por otro, en la representación lógica con fluents (Subsección 3.4.2) todos los hechos verdaderos están a la vista de todos, es decir, no hay fluents privados ($\mathcal{F}_{\text{privados}} = \emptyset$) y cualquier proposición verdadera en el estado actual es de conocimiento común.

Además, la búsqueda se vuelve determinista y sin incertidumbre. Los algoritmos de decisión del agente (Minimax, poda alfa-beta o Monte Carlo Tree Search) pueden simular directamente transiciones de un estado s a su sucesor $s' = T(s, \mathbf{a})$, y así anticipar las consecuencias de cada jugada explorando un árbol de estados determinista, sin tener que mantener árboles de conjuntos de información ni modelos parcialmente observables (POMDPs).

3.3. General Game Playing

Un General Game Player (GGP) es un agente que recibe la descripción formal de un juego y debe jugarlo sin ayuda humana ni conocimiento específico programado de antemano para ese dominio. Genesereth, Love y Pell definen el área como el estudio de sistemas capaces de aceptar la descripción formal de un juego y jugarlo bien sin depender de algoritmos diseñados para juegos particulares [8].

Lo que separa a un motor especializado de un General Game Player es dónde vive el conocimiento del juego. En el motor especializado, las reglas y heurísticas suelen estar dentro del código fuente. En un GGP, las reglas llegan como entrada externa, y el agente debe interpretarlas, construir su propia representación interna y aplicar un proceso de decisión general. Por eso la generalidad no consiste solo en poder ejecutar dos juegos: el mismo núcleo algorítmico tiene que funcionar cuando cambia la descripción del juego.

De aquí se sigue algo para este Trabajo Terminal: no queremos un motor de Ajedrez que rivalice con los sistemas especializados, sino una arquitectura desacoplada. Lo que hay que

demostrar es que el sistema carga reglas externas, genera movimientos legales, reconoce estados terminales y escoge jugadas con un criterio general de búsqueda. Y la prueba de que es general está en cambiar el archivo de reglas sin tocar el motor.

3.4. Modelo formal de juego

En la literatura de *General Game Playing* [8]-[10], un juego se formaliza como un sistema de transición de estados determinista, síncrono y multipartito. Visto así, una partida es la trayectoria que recorre un autómata finito por su espacio de configuraciones, impulsada por las decisiones simultáneas de varios agentes racionales.

Definición 3.4: Juego en General Game Playing

Un juego formal Γ es un sistema de transición de estados finito, determinista, síncrono y de información perfecta caracterizado por una tupla de ocho componentes [8]-[10]:

$$\Gamma = \langle R, S, s_0, \{A_i\}_{i=1}^{n}, \{L_i\}_{i=1}^{n}, T, F, \{G_i\}_{i=1}^{n} \rangle$$

donde:

- $R = \{r_1, r_2, \ldots, r_n\}$ es el conjunto finito de roles o participantes ($|R| = n \geq 1$).
- S es el conjunto finito y no vacío de estados posibles del entorno.
- $s_0 \in S$ es el estado inicial distinguido en el instante $t = 0$.
- A_i es el catálogo finito de acciones primitivas del participante r_i, conformando el espacio conjunto $\mathbf{A} = \prod_{i=1}^{n} A_i$.
- $L_i : S \to \mathcal{P}(A_i)$ es la función de legalidad que asigna a cada estado las acciones permitidas para el participante r_i.
- $T : S \times \mathbf{A} \to S$ es la función de transición dinámica que mapea el estado actual y un vector de acciones conjuntas a un único estado sucesor.
- $F \subseteq S$ es el subconjunto de estados terminales.
- $G_i : S \to [0, 100]$ es la función de utilidad o meta que asigna la recompensa numérica alcanzada por el rol r_i.

A continuación se detalla la semántica y formalización de cada uno de los componentes de la tupla.

3.4.1. Conjunto de roles (R)

El conjunto $R = \{r_1, r_2, \ldots, r_n\}$ es finito y no vacío, con cardinalidad $|R| = n \geq 1$, y sus elementos son los participantes del juego. En *General Game Playing*, esos elementos

no tienen para el motor de razonamiento ningún significado previo (“humano”, “blancas” o “máquina”); son identificadores abstractos. En el Gato, por ejemplo, $R = \{\text{x}, \text{o}\}$, y en el Ajedrez, $R = \{\text{blancas}, \text{negras}\}$. En GDL cada participante se declara con el predicado `(role r)`.

3.4.2. Espacio de estados (S) y semántica de fluents

El espacio de estados $S = \{s_1, s_2, \ldots, s_k\}$ es un conjunto finito y no vacío con todas las configuraciones admisibles del sistema. En lugar de tratar cada estado como un identificador opaco e indivisible, en *General Game Playing* se descompone en proposiciones estructuradas, a partir del concepto de *fluent* [9], [31].

Definición 3.5: Fluent

Un *fluent* es una proposición lógica atómica (indivisible) $f \in \mathcal{F}$ —concepto originado en el cálculo de situaciones por McCarthy y Hayes [31] y adoptado como base formal en GGP [9], [10]—, cuyo valor de verdad no es estático ni constante, sino una función dependiente del tiempo o del estado del sistema:

$$\text{val}(f, t) \in \{\text{Verdadero}, \text{Falso}\}$$

A diferencia de las relaciones estáticas del dominio, que valen igual durante toda la partida (las dimensiones del tablero o las reglas de adyacencia), un *fluent* representa un hecho contingente del entorno: puede aparecer, mantenerse o desaparecer según las transiciones que provocan las acciones ejecutadas.

Definición 3.6: Estado

En un entorno determinista y de información perfecta, el estado de un juego en el instante t se define lógicamente como la conjunción lógica ($\bigwedge$) de todos los *fluents* atómicos que son verdaderos simultáneamente en ese instante:

$$s_t \equiv \bigwedge_{i=1}^{m} f_i = f_1 \wedge f_2 \wedge \cdots \wedge f_m$$

Bajo la perspectiva conjuntista de la programación lógica declarativa y el principio de mundo cerrado (*Closed World Assumption*) formalizado por Reiter [32], el estado se modela de forma compacta como el subconjunto $s \subseteq \mathcal{F}$ de hechos atómicos verdaderos en dicho instante [9], [10]. En consecuencia, el espacio global de estados satisface:

$$S \subseteq \mathcal{P}(\mathcal{F})$$

donde cualquier *fluent* $f \notin s$ se evalúa formalmente como falso.

Ejemplo ilustrativo en el juego del Gato

Para ver esta semántica en un caso concreto, tomemos el juego del Gato (*Tic-Tac-Toe*) sobre una retícula de 3×3.

El universo de fluents $\mathcal{F}$ tiene dos clases de proposiciones atómicas. Las primeras describen la ocupación de cada casilla: $\mathcal{F}_{\text{celdas}} = \{\text{cell}(r, c, m) \mid r \in \{1,2,3\},\ c \in \{1,2,3\},\ m \in \{\text{x}, \text{o}, \text{b}\}\}$, donde b indica una casilla vacía (*blank*); son $3 \times 3 \times 3 = 27$ átomos posibles. Las segundas indican a quién le toca jugar: $\mathcal{F}_{\text{turno}} = \{\text{control}(\text{x}), \text{control}(\text{o})\}$, que son 2 átomos. Así, $\mathcal{F} = \mathcal{F}_{\text{celdas}} \cup \mathcal{F}_{\text{turno}}$, con $|\mathcal{F}| = 29$.

Al empezar la partida, las nueve casillas están vacías y el turno es de x. El estado inicial s_0 es entonces la conjunción de diez fluents verdaderos:

$$s_0 \equiv \left(\bigwedge_{r=1}^{3} \bigwedge_{c=1}^{3} \text{cell}(r, c, \text{b}) \right) \wedge \text{control}(\text{x}) \tag{3.10}$$

o, en su representación como conjunto:

$$s_0 = \{\text{cell}(1,1,\text{b}), \text{cell}(1,2,\text{b}), \ldots, \text{cell}(3,3,\text{b}), \text{control}(\text{x})\} \tag{3.11}$$

Proposiciones como $\text{cell}(2,2,\text{x})$ no pertenecen a s_0 y, por lo tanto, son falsas en $t = 0$.

Si x marca el centro con la acción $\text{mark}(2,2)$, la función de transición quita los fluents invalidados y agrega los nuevos:

$$s_1 = \big(s_0 \setminus \{\text{cell}(2,2,\text{b}), \text{control}(\text{x})\}\big) \cup \{\text{cell}(2,2,\text{x}), \text{control}(\text{o})\} \tag{3.12}$$

Como conjunción, la nueva configuración s_1 queda así:

$$s_1 \equiv \text{cell}(2,2,\text{x}) \wedge \text{control}(\text{o}) \wedge \bigwedge_{(r,c) \neq (2,2)} \text{cell}(r, c, \text{b}) \tag{3.13}$$

Las ocho celdas vacías restantes se mantienen igual por inercia.

En GDL, para preguntar si un *fluent* vale en el estado actual se usa `(true f)`. De cómo empieza el juego y de cómo cambia se ocupan `(init f)` y `(next f)`; más adelante, en los listados del Gato, se ven los tres en acción [9], [10].

3.4.3. Estado inicial (s_0)

El elemento distinguido $s_0 \in S$ es la configuración del entorno en el instante $t = 0$, antes de que los agentes tomen ninguna decisión. En una descripción declarativa, s_0 se construye con todos los hechos que se declaran mediante el predicado `(init f)`.

3.4.4. Acciones individuales (A_i) y espacio conjunto de acciones (A)

Para cada rol $r_i \in R$, el conjunto finito $A_i = \{a_{i,1}, a_{i,2}, \ldots, a_{i,m_i}\}$ reúne las acciones primitivas que ese participante puede concebir. Como el modelo es síncrono, en cada paso de tiempo todos

los jugadores eligen una acción a la vez, y el espacio global de decisiones es el producto cartesiano de los conjuntos individuales:

$$\mathbf{A} = \prod_{i=1}^{n} A_i = A_1 \times A_2 \times \cdots \times A_n \tag{3.14}$$

Un elemento $\mathbf{a} = (a_1, a_2, \ldots, a_n) \in \mathbf{A}$ es una acción conjunta, un vector de decisiones concurrentes en el que cada componente $a_i \in A_i$ es la jugada del rol r_i.

3.4.5. Función de legalidad (L_i)

Que una acción sea válida depende del estado actual del sistema. La legalidad de cada jugador r_i se modela con la correspondencia:

$$L_i : S \to \mathcal{P}(A_i) \tag{3.15}$$

que asigna a cada configuración $s \in S$ el subconjunto de acciones permitidas $L_i(s) \subseteq A_i$. En su formulación relacional original [8], [9], el criterio se expresa como una relación binaria $L_i \subseteq A_i \times S$, en la que $(a, s) \in L_i$ significa que la acción a se puede ejecutar en s; corresponde directamente al predicado `(legal r a)` [10].

Una acción conjunta $\mathbf{a} = (a_1, \ldots, a_n)$ es legal en el estado s si y solo si cada componente es legal para su rol:

$$\mathbf{a} \in \mathbf{L}(s) \iff \forall i \in \{1, \ldots, n\}, \quad a_i \in L_i(s) \tag{3.16}$$

donde $\mathbf{L}(s) = \prod_{i=1}^{n} L_i(s)$ es el espacio de jugadas conjuntas válidas en ese estado.

3.4.6. Función de transición dinámica (T)

La evolución del juego en el tiempo la rige la función de transición determinista:

$$T : S \times \mathbf{A} \to S \tag{3.17}$$

que determina de forma única el estado sucesor $s' = T(s, \mathbf{a})$ que resulta de aplicar la acción conjunta $\mathbf{a} \in \mathbf{L}(s)$ sobre la configuración s. Dos propiedades la caracterizan. Es estrictamente determinista: a diferencia de los procesos estocásticos o del control continuo, evaluar las reglas produce un único estado resultante y no hay incertidumbre en la física del sistema. Y es reactiva y síncrona: el universo cambia solo como consecuencia de las decisiones conjuntas de los agentes, algo que en GDL se especifica con las reglas de deducción de hechos futuros `(next f)`.

3.4.7. Conjunto de estados terminales (F)

El subconjunto $F \subseteq S$ marca las condiciones en las que termina una partida. Cuando el sistema llega a una configuración $s \in F$, la trayectoria se detiene: ya no se admiten más movimientos y se calculan los resultados finales. En los juegos de mesa, este conjunto incluye situaciones como el jaque mate, las tablas por repetición, una línea ganadora o el tablero lleno. En el motor lógico se verifica con el predicado `(terminal)`.

3.4.8. Función de utilidad o meta (G_i)

Para cada participante $r_i \in R$, la función de utilidad o pago:

$$G_i : S \to [0, 100] \tag{3.18}$$

Aunque formalmente se puede evaluar en cualquier estado, la función tiene sentido sobre todo en los estados terminales $s \in F$. Siguiendo el estándar de GGP, los valores se normalizan en la escala discreta $[0, 100]$: 100 es victoria, 0 derrota y 50 empate.

En los juegos adversariales de dos participantes y suma constante, como el Ajedrez y el Gato, las utilidades terminales cumplen:

$$G_1(s) + G_2(s) = 100, \quad \forall s \in F \tag{3.19}$$

lo cual asegura el carácter competitivo de suma constante que exigen algoritmos como Minimax. Esta función se calcula con la relación lógica `(goal r v)`.

3.4.9. Axiomas de admisibilidad y juegos bien formados

Para que un agente autónomo pueda resolver computacionalmente un juego formalizado como máquina de estados, se piden dos propiedades estructurales [8], [9]:

Definición 3.7: Juego bien formado

Un juego formal Γ se define como bien formado (*well-formed game*) [8]-[10] si satisface dos axiomas fundamentales:

1. **Jugabilidad (*Playability*):** Para todo estado no terminal, cada rol dispone de al menos una acción legal ejecutable:

   $$\forall s \in S \setminus F, \quad \forall r_i \in R : \quad L_i(s) \neq \emptyset$$

   Esta condición previene bloqueos indeterminados (*deadlocks*) en la ejecución algorítmica del motor.

2. **Terminación garantizada (*Termination*):** Toda trayectoria legal inducida a partir del estado inicial s_0:

   $$s_0 \xrightarrow{\mathbf{a}_0} s_1 \xrightarrow{\mathbf{a}_1} s_2 \ldots \xrightarrow{\mathbf{a}_{k-1}} s_k$$

   debe culminar forzosamente en un estado terminal $s_k \in F$ en un número finito de transiciones ($k < \infty$).

En dominios donde podrían presentarse ciclos infinitos, la terminación se asegura con reglas de repetición o con cotas explícitas de turnos.

3.4.10. Distancia en el grafo de estados y estimación hacia configuraciones terminales

La relación entre el conjunto de estados terminales F y el axioma de terminación permite definir una noción de distancia en el espacio de estados. En el grafo dirigido que induce la

máquina de estados $\Gamma = \langle R, S, s_0, \{A_i\}_{i=1}^n, \{L_i\}_{i=1}^n, T, F, \{G_i\}_{i=1}^n \rangle$, una trayectoria de juego es una secuencia finita de transiciones legales dadas por la función $T(s, \mathbf{a})$. Con esta estructura se puede formalizar la distancia entre dos estados cualesquiera:

Definición 3.8: Distancia dirigida entre estados

En un juego formal con espacio de estados S, función de transición determinista $T : S \times \mathbf{A} \to S$ y función de legalidad conjunta $\mathbf{L} : S \to \mathcal{P}(\mathbf{A})$, adaptando la formulación de trayectorias en espacios de estados de planificación heurística [1], [33], la **distancia en el grafo de estados** $d(s_a, s_b)$ entre una configuración origen $s_a \in S$ y una configuración destino $s_b \in S$ se define como el número mínimo de acciones conjuntas legales requeridas para transitar de s_a a s_b:

$$d(s_a, s_b) = \min \left\{ k \in \mathbb{N} \;\middle|\; \exists \mathbf{a}_0, \ldots, \mathbf{a}_{k-1} : s^{(0)} = s_a,\ s^{(j+1)} = T(s^{(j)}, \mathbf{a}_j),\ s^{(k)} = s_b \right\}$$

donde cada vector de decisión es admisible en su respectivo estado ($\mathbf{a}_j \in \mathbf{L}(s^{(j)})$ para todo $j \in \{0, \ldots, k-1\}$). En caso de que no exista ninguna trayectoria legal originada en s_a que conduzca a s_b, la distancia adopta convencionalmente el valor $d(s_a, s_b) = \infty$.

Sobre la notación: el superíndice entre paréntesis $s^{(j)}$ no es una potencia de s, sino el **índice temporal discreto**, es decir, el número de paso a lo largo de la trayectoria. Se usan paréntesis justamente para no confundirlo con un exponente: $s^{(0)} = s_a$ es la configuración inicial, $s^{(1)} = T(s^{(0)}, \mathbf{a}_0)$ el estado tras la primera jugada, $s^{(j)}$ el estado intermedio después de j transiciones y $s^{(k)} = s_b$ el estado final tras k decisiones conjuntas.

Con esta definición se puede obtener la **distancia al estado terminal más cercano** desde una configuración s:

$$d(s, F) = \min_{s_f \in F} d(s, s_f) \tag{3.20}$$

De aquí se sigue directamente, por el axioma de terminación finita, que para toda configuración alcanzable en una partida legítima ($s \in \text{Reach}(s_0)$) la distancia al conjunto terminal F es finita y está acotada por el horizonte máximo del juego:

$$\forall s \in \text{Reach}(s_0) : \quad 0 \leq d(s, F) \leq H_{\text{máx}} < \infty \tag{3.21}$$

y $d(s, F) = 0$ si y solo si la configuración evaluada ya es un estado terminal ($s \in F$).

Para la toma de decisiones, el conjunto terminal F no es homogéneo: se divide en clases disjuntas según la utilidad $G_i(s)$ de cada participante. En un juego adversarial de suma constante, un agente que juega con el rol r_i distingue dos regiones dentro de F: los estados de victoria $F_{\text{victoria}} = \{s \in F \mid G_i(s) = 100\}$ y los de derrota $F_{\text{derrota}} = \{s \in F \mid G_i(s) = 0\}$. Con esa división se puede medir qué tan cerca está el estado actual de cada desenlace:

$$d(s, F_{\text{victoria}}) = \min_{s_f \in F_{\text{victoria}}} d(s, s_f) \tag{3.22}$$

$$d(s, F_{\text{derrota}}) = \min_{s_f \in F_{\text{derrota}}} d(s, s_f) \tag{3.23}$$

Los algoritmos de búsqueda adversarial como Minimax buscan maximizar la utilidad terminal esperada $G_i(s)$ del agente, suponiendo que el adversario elegirá las jugadas que la minimicen

(principio de inducción hacia atrás):

$$V(s) = \max_{a_i \in L_i(s)} \min_{a_{-i} \in L_{-i}(s)} V\big(T(s, (a_i, a_{-i}))\big) \tag{3.24}$$

donde en los nodos terminales $s \in F$ el valor coincide con la utilidad, $V(s) = G_i(s)$. Esta formulación se relaciona con las distancias $d(s, F_{\text{victoria}})$ y $d(s, F_{\text{derrota}})$ de dos maneras.

La primera es como criterio de desempate. Cuando varias ramas llevan a una victoria segura ($G_i(s) = 100$), un agente racional prefiere la acción que minimice $d(s, F_{\text{victoria}})$: gana en menos transiciones y le da menos margen de réplica al adversario. Si la derrota es inevitable ($G_i(s) = 0$), busca lo contrario, maximizar $d(s, F_{\text{derrota}})$, para alargar el final y dar oportunidad a que el rival se equivoque.

La segunda es como heurística sustituta en horizontes finitos. En dominios complejos, la profundidad del juego impide llegar a las hojas terminales y $G_i(s)$ no está disponible en los nodos frontera. Ahí, la cercanía a los subconjuntos terminales, o las cotas calculadas sobre el espacio de *fluents*, funciona como una función heurística que estima la probabilidad o expectativa de ganar y orienta la búsqueda hacia las regiones favorables del espacio de estados.

Sin embargo, en dominios con gran complejidad combinatoria como el Ajedrez, calcular estas distancias de forma exacta con una exploración exhaustiva, por ejemplo una búsqueda en anchura sobre el grafo de estados, es inviable por la explosión exponencial del espacio de búsqueda. Para sortear el obstáculo, en General Game Playing se aprovecha la estructura interna de los *fluents* y se construyen **cotas inferiores admisibles** que pueden evaluarse rápido durante la ejecución.

Supongamos que una condición terminal de victoria $\mathcal{M} \subset \mathcal{F}$ exige que se cumplan a la vez varios hechos proposicionales (en el Gato, tres fichas juntas en una línea). Se puede contar cuántos de esos *fluents* aún no se han materializado en el estado actual:

$$\text{Faltantes}(s, \mathcal{M}) = \{f \in \mathcal{M} \mid f \notin s\} \tag{3.25}$$

La dinámica del juego limita cuántos hechos meta se pueden agregar en una sola transición. Si llamamos $c \in \mathbb{N}$ a esa cota de producción (típicamente $c = 1$ cuando cada jugador añade a lo sumo una marca o pieza por movimiento), la distancia real al estado terminal tiene una cota inferior analítica:

$$d(s, s_{\mathcal{M}}) \geq \left\lceil \frac{|\text{Faltantes}(s, \mathcal{M})|}{c} \right\rceil \tag{3.26}$$

En el Gato, si una configuración s ya tiene dos marcas de x en una misma fila, al conjunto de hechos faltantes para completar la línea solo le queda un elemento ($|\text{Faltantes}| = 1$). Como $c = 1$, la cota inferior garantiza que la victoria exige al menos una jugada propia, y eso le permite al agente explorar primero esa alternativa antes que otras ramas tácticas secundarias.

En los motores avanzados de GGP [20], [21], este principio se extiende con la **relajación de efectos negativos** (*delete relaxation*), una técnica que Bonet y Geffner formularon en planificación heurística [33] y que después se adaptó al razonamiento heurístico general. Consiste en ignorar de manera temporal las reglas que destruyen o invalidan *fluents*, suponiendo de forma optimista que los hechos verdaderos nunca se eliminan ni se sobrescriben. Con ese supuesto, encontrar la trayectoria más corta hacia cualquier configuración terminal de F deja de ser intratable y se resuelve en tiempo polinomial. La longitud del camino calculado en el espacio relajado sirve como heurística informada y admisible, y permite estimar numéricamente qué tan cerca está el final de la partida sin usar funciones de evaluación hechas a mano para un juego en particular.

3.4.11. Modelado síncrono de juegos por turnos alternados

Aunque el producto cartesiano $\mathbf{A} = A_1 \times A_2$ supone que las decisiones son simultáneas, los juegos típicamente secuenciales, como el Gato o el Ajedrez, encajan en este esquema mediante una acción pasiva o nula, noop (*no-operation*) [9], [10].

En un estado s donde le corresponde mover al jugador blanco, la función de legalidad asigna:

$$L_{\text{blancas}}(s) \subseteq A_{\text{blancas}} \setminus \{\text{noop}\} \tag{3.27}$$

$$L_{\text{negras}}(s) = \{\text{noop}\} \tag{3.28}$$

y la única acción conjunta admisible es el par ordenado:

$$\mathbf{a} = (a_{\text{blanca}}, \text{noop}) \tag{3.29}$$

Al calcular la transición $s' = T(s, \mathbf{a})$, las reglas del juego actualizan los fluents de control y se invierte la legalidad en la siguiente configuración:

$$L_{\text{blancas}}(s') = \{\text{noop}\} \tag{3.30}$$

$$L_{\text{negras}}(s') \subseteq A_{\text{negras}} \setminus \{\text{noop}\} \tag{3.31}$$

Así la máquina de estados conserva una arquitectura síncrona homogénea sin perder la alternancia secuencial propia del dominio.

3.4.12. Interfaz computacional para el motor de decisión

Para que esta formalización pueda usarse en la arquitectura de software del agente, el motor de juego debe ofrecer una interfaz programática con cinco consultas básicas: obtener el estado inicial (s_0); consultar las acciones legales de un jugador en un estado dado ($L_i(s)$); aplicar una acción conjunta y calcular el estado sucesor ($T(s, \mathbf{a})$); determinar si un estado cumple la condición de terminación ($s \in F$); y evaluar la utilidad que obtiene cada participante ($G_i(s)$).

Con esta interfaz, el motor de búsqueda queda completamente separado de las particularidades de cada juego. El algoritmo de decisión razona solo sobre estados abstractos, acciones legales y recompensas numéricas, sin conocimiento empírico propio del dominio.

3.5. Lenguaje de descripción de juegos (GDL)

Para que un agente pueda jugar sin que nadie le explique las reglas, estas tienen que estar escritas en un lenguaje que él mismo pueda leer, analizar y ejecutar. Eso es un lenguaje de descripción de juegos. En *General Game Playing*, el de referencia es el *Game Description Language* (GDL), desarrollado en la Universidad de Stanford [8], [10].

En un lenguaje imperativo como C++ o Python, la lógica de un juego se programa con estructuras de control, bucles y modificaciones destructivas de la memoria: se ordena qué hacer paso a paso. Un lenguaje de descripción de reglas va por otro lado, el declarativo, y en lugar de decir cómo se ejecuta el juego, dice qué es.

3.5.1. Características fundamentales de GDL

A partir de los fundamentos de Genesereth et al. [8], Love et al. [10] y Cropper et al. [34], pueden destacarse cuatro propiedades de GDL.

La primera es que se basa en lógica de primer orden y es declarativo. El autor del juego no programa cómo se ejecutan las jugadas: solo declara qué hechos son verdaderos y bajo qué condiciones cambian, y es el motor de inferencia del agente el que deduce las consecuencias de esas reglas. La segunda es su independencia del dominio. GDL no trae ideas previas sobre geometría de tableros, piezas de ajedrez o cartas; todo se representa con símbolos abstractos, y un agente general aprende la física y las restricciones del entorno mientras procesa el archivo de reglas, porque no las conoce de antemano.

La tercera es su semántica, basada en Datalog con negación estratificada. GDL es una variante estricta de Datalog que limita el uso de funciones recursivas complejas, lo que garantiza que el espacio de estados y la deducción lógica sean finitos y decidibles en cada paso de tiempo, y evita ciclos de inferencia infinitos. La cuarta es el desacoplamiento arquitectónico: la física del juego, descrita en el archivo de reglas, queda totalmente separada de los algoritmos de búsqueda y decisión, que viven en el motor del agente. Por eso el mismo núcleo puede jugar juegos muy distintos sin cambiar una línea de código fuente.

3.5.2. Concepto y estructura de un predicado lógico

Para entender la sintaxis de GDL sin caer en un formalismo matemático excesivo, conviene partir de la experiencia del desarrollo de software:

Definición 3.9: Predicado lógico

En la semántica lógica declarativa de GDL [9], [10], un **predicado** es una afirmación estructurada sobre uno o varios objetos del dominio que, al ser evaluada contra el estado actual del juego, devuelve estrictamente un valor de verdad: verdadero o falso.

Un programador que quiere saber si hay una cruz en el renglón 1 y la columna 1 escribiría algo como `tablero[1][1] == 'x'`: mirar una celda de una matriz. GDL no consulta nada, afirma un hecho. La misma pregunta se escribe

```
(cell 1 1 x)
```

y la respuesta es simplemente si ese hecho es verdadero o falso en el estado actual.

Un predicado se escribe como una lista entre paréntesis y tiene tres partes. La primera es el *functor*, el identificador inicial que dice qué propiedad o relación se expresa (por ejemplo, `cell`, `control`, `legal`, `next`). La segunda son los términos o argumentos, los elementos sobre los que recae la afirmación. Estos pueden ser constantes, símbolos atómicos que denotan objetos concretos del dominio, como coordenadas (`1`, `2`, `3`) o identificadores (`x`, `o`, y `b` para una casilla vacía o *blank*); o variables, que en GDL son cualquier símbolo que empieza con un signo de interrogación (`?r` para el renglón, `?c` para la columna, `?p` para el participante) y permiten escribir reglas generales válidas para cualquier casilla o jugador.

La tercera parte es la aridad, el número de argumentos que acepta el predicado. Con aridad 0 tenemos una proposición pura, como `(terminal)`, que no recibe argumentos y expresa un hecho

global: si la partida terminó. Con aridad 1, una propiedad de un objeto, por ejemplo `(role x)` o `(control x)`, que indican que `x` es un rol o que tiene el turno. Con aridad 3, una relación ternaria como `(cell 2 2 x)`, que une renglón, columna y ficha.

Construcción de reglas de inferencia (<=)

Los hechos no siempre están dados de forma estática; muchas veces hay que deducirlos de otros hechos. Para eso, GDL usa reglas de inferencia con el operador de implicación <=, que se lee "la cabeza es verdadera si todas las premisas del cuerpo son verdaderas":

```
(<= (cabeza_deducida)
    (premisa_o_condicion_1)
    (premisa_o_condicion_2)
    ...
    (premisa_o_condicion_k))
```

Código 3.1: Estructura canónica de una regla de inferencia en GDL.

El motor lógico solo da por cierta la cabeza si logra verificar que todas las condiciones del cuerpo se cumplen a la vez en el estado del juego.

3.5.3. Predicados reservados del sistema en GDL

Para que un archivo de reglas describa la tupla formal Γ de la Sección 3.4, GDL define un conjunto cerrado de ocho predicados del sistema [9], [10], que se resumen en la Tabla 3.1.

Tabla 3.1: Predicados reservados de GDL y componente de Γ que describen.

Predicado	Función
`(role ?r)`	Declara a los participantes del juego e instancia el conjunto R.
`(init ?f)`	Especifica los fluents atómicos que forman el estado inicial s_0.
`(true ?f)`	Consulta si un fluent dinámico f es verdadero en el estado actual s.
`(does ?r ?a)`	Representa la acción $a \in A_i$ que el rol r ejecuta en el turno corriente.
`(legal ?r ?a)`	Deduce si la acción a pertenece al conjunto de jugadas permitidas $L_i(s)$.
`(next ?f)`	Deduce qué fluents formarán parte del estado sucesor $s' = T(s, \mathbf{a})$.
`(terminal)`	Evalúa si la configuración actual pertenece al conjunto de estados terminales F.
`(goal ?r ?v)`	Determina la utilidad o recompensa numérica $G_i(s) = v \in [0, 100]$ del jugador r.

3.5.4. Ejemplificación mediante código de especificación: El juego del Gato

Para ver cómo trabajan juntos estos predicados, tomemos la especificación del juego del Gato (*Tic-Tac-Toe*), que se puede dividir en cuatro bloques lógicos.

Roles y estado inicial

El primer bloque (Código 3.2) declara los dos jugadores y describe el tablero al inicio ($t = 0$): las nueve casillas vacías (`b`) y el turno en manos de `x`.

```
;; ========================================
;; 1. Declaracion de roles (Jugadores)
;; ========================================
(role x)
(role o)

;; ========================================
;; 2. Estado inicial (t = 0):
;;    Nueve casillas vacias (b = blank)
;;    y control asignado a x
;; ========================================
(init (cell 1 1 b))
(init (cell 1 2 b))
(init (cell 1 3 b))
(init (cell 2 1 b))
(init (cell 2 2 b))
(init (cell 2 3 b))
(init (cell 3 1 b))
(init (cell 3 2 b))
(init (cell 3 3 b))
(init (control x))
```

Código 3.2: Declaración de roles y estado inicial para el juego del Gato.

Reglas de legalidad y alternancia de turnos

El segundo bloque (Código 3.3) formaliza la función de legalidad $L_i(s)$. Una jugada es legal si quien la intenta tiene el control y la casilla elegida está libre. La sincronía se modela con la jugada nula noop:

```
;; ========================================
;; 3. Acciones legales
;; ========================================
;; Un jugador puede marcar (?r, ?c) si es su turno
;; y la casilla actualmente contiene 'b'
(<= (legal ?p (mark ?r ?c))
    (true (control ?p))
    (true (cell ?r ?c b)))

;; Si NO es el turno del jugador, su unica accion legal es 'noop'
(<= (legal ?p noop)
    (role ?p)
    (not (true (control ?p))))
```

Código 3.3: Reglas de legalidad de movimientos en GDL.

Con estas reglas el motor rechaza por sí solo cualquier intento de poner una ficha en una casilla ocupada, y al rival que no tiene el turno le deja únicamente la acción nula.

Transición de estado y persistencia inercial

El tercer bloque (Código 3.4) gobierna la función de transición $T(s, \mathbf{a})$. En lógica declarativa, un hecho no se conserva en el estado siguiente a menos que una regla lo diga de forma explícita, así que hay que definir tanto los cambios directos como la inercia del sistema:

```
;; ========================================
;; 4. Funcion de transicion (Siguiente estado)
;; ========================================
;; A. Efecto directo: la casilla marcada pasa a tener la ficha del jugador
(<= (next (cell ?r ?c ?p))
    (does ?p (mark ?r ?c)))

;; B. Inercia de fichas: las celdas ya marcadas conservan su contenido
(<= (next (cell ?r ?c ?mark))
    (true (cell ?r ?c ?mark))
    (distinct ?mark b))

;; C. Inercia de celdas vacias: una casilla sigue vacia si nadie la marco
(<= (next (cell ?r ?c b))
    (true (cell ?r ?c b))
    (not (does x (mark ?r ?c)))
    (not (does o (mark ?r ?c))))

;; D. Alternancia del turno
(<= (next (control o)) (true (control x)))
(<= (next (control x)) (true (control o)))
```

Código 3.4: Reglas de transicion dinamica e inercia en GDL.

Condiciones terminales y asignación de recompensas

El cuarto bloque (Código 3.5) define el conjunto de estados terminales F y la función de utilidad $G_i(s)$: premia la formación de líneas de tres fichas y reconoce el empate cuando el tablero se llena:

```
;; ========================================
;; 5. Condiciones de victoria y utilidades
;; ========================================
;; Victoria absoluta (100 puntos) si el jugador forma una linea
(<= (goal ?p 100) (line ?p))

;; Derrota (0 puntos) si el adversario completo una linea
(<= (goal ?p 0)   (role ?p) (line ?other) (distinct ?p ?other))

;; Empate (50 puntos) si el tablero esta lleno y nadie gano
(<= (goal ?p 50)  (role ?p) (not (line x)) (not (line o)) board_full)

;; ========================================
;; 6. Condicion de fin de partida (terminal)
;; ========================================
(<= terminal (line x))
(<= terminal (line o))
```

```
18 (<= terminal board_full)
```

Código 3.5: Reglas de utilidad (goal) y condicion de terminacion en GDL.

Este ejemplo muestra qué tan expresivo puede ser un lenguaje de descripción de juegos: en menos de cincuenta líneas declarativas, y sin una sola instrucción imperativa de control, el archivo describe completo el universo de reglas del Gato.

3.6. GDL, Ludii y lenguajes de dominio específico

Describir un juego con GDL clásico es una opción entre varias. Otros lenguajes intentan que la escritura sea más cómoda sin perder expresividad ni volverse difíciles de implementar. Ludii es un caso claro: en vez de predicados lógicos habla de tableros, piezas y movimientos, más cerca de cómo piensa quien diseña juegos [26].

Comparar GDL con Ludii importa para este Trabajo Terminal porque obliga a tomar una decisión de arquitectura: implementar GDL completo, adoptar solo un subconjunto o diseñar un lenguaje de dominio específico (DSL) inspirado en ambos. GDL tiene una base académica sólida y una semántica lógica general, pero puede resultar pesado para un prototipo de alcance acotado. Ludii, por su parte, demuestra que un lenguaje puede seguir siendo general y ser más legible para quienes diseñan y desarrollan juegos.

Un DSL propio permitiría concentrarse en lo que el proyecto necesita: jugadores, estados, acciones, reglas de legalidad, transiciones, condiciones terminales y recompensas. Esta opción tendrá que justificarse formalmente, para que el lenguaje no termine siendo una colección de casos especiales. El criterio de diseño es que permita describir los juegos dentro del alcance definido: discretos, deterministas, por turnos y de información perfecta.

3.7. Intérprete y representación interna

Tener el archivo de reglas no basta para jugar. Hace falta un intérprete que lea la descripción, valide su estructura y construya una representación interna que el motor del agente pueda consultar. El proceso suele pasar por análisis léxico, análisis sintáctico, validación semántica y construcción de un modelo interno.

El análisis léxico separa el texto en unidades reconocibles y el sintáctico comprueba que esas unidades respeten la gramática del lenguaje. La validación semántica revisa que las reglas tengan sentido dentro del modelo de juego; por ejemplo, que las acciones hagan referencia a jugadores que existen o que una condición terminal pueda evaluarse. Al final, la representación interna organiza toda esa información para responder consultas durante la partida.

La representación interna hace de puente entre el lenguaje textual y los algoritmos de decisión. Una vez construido el modelo, el motor ya no debería trabajar con cadenas de texto, sino con estructuras que permitan evaluar legalidad, transiciones y recompensas. Esto ayuda al mantenimiento: en principio se podría cambiar el lenguaje textual sin rehacer el motor de búsqueda, siempre que se conserve la interfaz conceptual del juego.

En las implementaciones avanzadas de GGP, las reglas también pueden compilarse en estructuras más eficientes. Goldwaser y Thielscher señalan que una descripción GDL puede

convertirse en una red proposicional (PropNet), con nodos lógicos, de entrada, de salida y de transición [28]. Esto interesa porque interpretar las reglas desde cero en cada nodo de búsqueda resulta caro. Una PropNet completa quizá quede fuera de una primera versión, pero la idea sirve para entender por qué la eficiencia de la representación interna pesa tanto como el algoritmo de búsqueda.

3.8. Árboles de juego y explosión combinatoria

Con acciones legales y transiciones ya se puede armar un árbol de juego [1]: la raíz es el estado actual, de ella salen tantas ramas como acciones legales haya, y cada rama termina en el estado que produce esa acción. Si el proceso se repite, el agente ve hacia adelante y puede comparar a dónde lleva cada decisión.

El problema es que el número de nodos crece muy rápido. Si un estado tiene un factor de ramificación promedio de b acciones y se explora hasta la profundidad d, el número de nodos puede crecer del orden de b^d. En juegos pequeños es manejable; en juegos complejos, explorarlo todo no es práctico.

Por la explosión combinatoria hay que limitar la búsqueda: el agente debe decidir cuánto explorar, qué ramas priorizar y cómo evaluar los estados que no son terminales. Aquí entran Minimax, la poda alfa-beta y Monte Carlo Tree Search, junto con funciones heurísticas que estiman qué tan bueno es un estado sin llegar hasta el final de la partida.

3.9. Minimax

Minimax es un algoritmo clásico para juegos adversariales de dos jugadores. Se apoya en el teorema minimax de von Neumann [35] y fue formulado para decidir en árboles de juego por Shannon [11] y, más tarde, por Russell y Norvig [1]. La idea básica es suponer que un jugador trata de maximizar su utilidad mientras que el oponente trata de minimizarla, de modo que en el árbol los niveles se alternan entre decisiones de MAX y de MIN.

Definición 3.10: Función de valor Minimax y decisión óptima

En un juego formal de dos participantes ($R = \{r_1, r_2\}$) determinista, de información perfecta y suma constante [1], [11], [35], el **valor Minimax** $V(s)$ de un estado $s \in S$ desde la perspectiva de MAX (r_1) se define recursivamente como:

$$V(s) = \begin{cases} G_1(s) & \text{si } s \in F \\ \max_{a \in L_1(s)} V(T(s, (a, \text{noop}))) & \text{si el turno corresponde a } r_1 \\ \min_{a \in L_2(s)} V(T(s, (\text{noop}, a))) & \text{si el turno corresponde a } r_2 \end{cases} \tag{3.32}$$

La **decisión óptima** $a^* \in L_i(s)$ para el participante activo corresponde a la acción que optimiza (maximiza para r_1, minimiza para r_2) el valor del estado sucesor inducido.

Si el árbol puede explorarse por completo hasta los estados terminales F, Minimax garantiza elegir una acción óptima siempre que ambos jugadores actúen racionalmente. El valor de un

estado terminal sale directo de la función de utilidad $G_1(s)$, y el de cualquier estado no terminal se calcula propagando hacia arriba los valores de sus sucesores: MAX toma el mayor y MIN el menor.

En la práctica, casi todos los juegos tienen factores de ramificación demasiado grandes para llegar a la frontera terminal. Por eso se fija una profundidad máxima de búsqueda y se evalúan los estados de la frontera con una función heurística. Así Minimax se puede usar en juegos más grandes, pero aparece la pregunta central para *General Game Playing*: cómo evaluar estados sin usar conocimiento del dominio programado de antemano.

3.10. Poda alfa-beta

La poda alfa-beta es una optimización básica de Minimax, analizada con detalle por Knuth y Moore [36] y descrita en la literatura clásica de inteligencia artificial [1]. Bajo las mismas condiciones de evaluación no cambia la decisión final, pero evita explorar las ramas que no pueden modificar el resultado. Para ello mantiene dos valores: alfa, la mejor alternativa que ha encontrado MAX, y beta, la mejor que ha encontrado MIN.

Cuando detecta que una rama no puede dar una decisión mejor que otra ya conocida, deja de explorarla. Así visita menos nodos y puede llegar a mayor profundidad con el mismo presupuesto de cómputo. Su efectividad depende del orden en que se exploren las acciones: si las mejores se revisan temprano, se descartan más ramas.

En este Trabajo Terminal, alfa-beta interesa porque permite comparar dos cosas: la decisión que produce Minimax y la eficiencia que se gana al podar. En la evaluación experimental, métricas como los nodos explorados, el tiempo de decisión y la profundidad alcanzada pueden mostrar si la optimización mejora el rendimiento sin afectar la legalidad ni la coherencia de las jugadas. La Figura 3.2 muestra cómo funciona el corte: en cuanto MIN encuentra una alternativa con valor menor o igual a la cota α que MAX ya tiene asegurada, se descartan de inmediato los demás sucesores de ese subárbol.

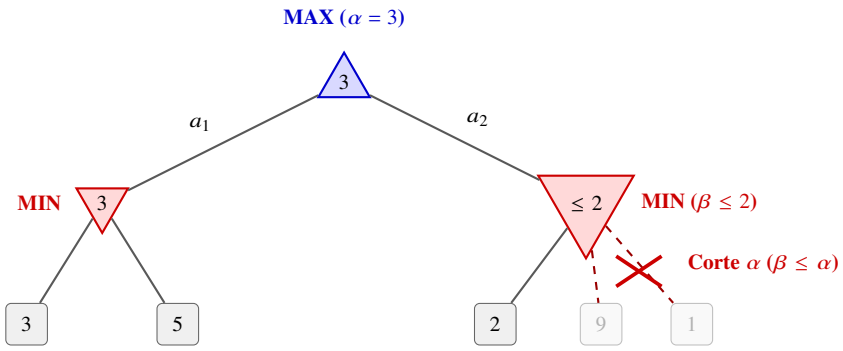


Figura 3.2: Árbol de juego adversarial con poda alfa-beta. Tras evaluarse el nodo con valor 2, el nodo MIN asegura un valor a lo sumo de 2, el cual es estrictamente inferior al valor $\alpha = 3$ ya garantizado por MAX en la rama izquierda; en consecuencia, se interrumpe la exploración del resto de los sucesores.

3.11. Funciones heurísticas generales

Una función heurística estima qué tan favorable es un estado cuando no se puede explorar el árbol hasta el final. En los motores especializados suele llevar conocimiento del dominio; en Ajedrez, por ejemplo, se puede dar un valor a cada pieza o medir el control del centro. Pero esas reglas no son generales: si el agente sabe cuánto vale una reina, ya trae conocimiento específico de Ajedrez.

En GGP el problema es evaluar sin saber de qué juego se trata. Se puede recurrir a rasgos estructurales: cuántas acciones legales hay (la movilidad), qué tan cerca se está de un estado terminal, cómo cambian las recompensas o qué resultados dan las simulaciones. Ninguno de estos rasgos funciona bien en todos los juegos; lo valioso es que ninguno presupone un dominio y, combinados, pueden aproximar una evaluación en un agente que no conoce el juego.

Aquí está uno de los puntos más delicados del proyecto. Limitar la búsqueda obliga a decidir qué estados parecen mejores, pero ser generales impide escribir esa evaluación para un juego concreto. De ahí que las heurísticas generales y los métodos basados en simulación ocupen un lugar central en la fundamentación teórica del motor de decisión.

3.12. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) es una familia de métodos de búsqueda que estima el valor de las acciones mediante simulaciones. En vez de depender solo de una función heurística explícita, va construyendo poco a poco un árbol de búsqueda y usa los resultados de partidas simuladas para orientar la exploración [19], [37].

MCTS funciona como un ciclo iterativo y asimétrico de cuatro fases que se repite mientras haya tiempo de cómputo, como se ve en la Figura 3.3. En la *selección*, el algoritmo baja por el árbol acumulado desde la raíz eligiendo los sucesores más prometedores con una política de balance. En la *expansión* se agregan uno o varios nodos hijos que corresponden a acciones legales aún no exploradas. En la *simulación*, o *rollout*, se juega una partida rápida con movimientos muestreados al azar hasta llegar a una configuración terminal. Por último, en la *retropropagación*, la recompensa terminal se propaga hacia la raíz y actualiza las estadísticas de visitas y el valor medio de cada estado del camino recorrido.

Lo que hace tentador a MCTS para GGP es que no exige heurísticas propias de un juego. Basta con un modelo capaz de dar las acciones legales, aplicar una transición y decir cuándo se terminó la partida; con eso se juegan partidas al azar y los resultados indican qué acciones convienen. El costo es que todo depende de qué tan rápido y bien esté hecho ese modelo, y de cuántas simulaciones alcancen en el tiempo disponible.

3.13. UCT

Una pregunta central en la fase de selección de MCTS es qué rama explorar. Si el agente explora solo la acción que hasta ese momento parece la mejor, corre el riesgo de ignorar alternativas poco visitadas que en realidad serían superiores. Si explora sin criterio, puede perder tiempo de

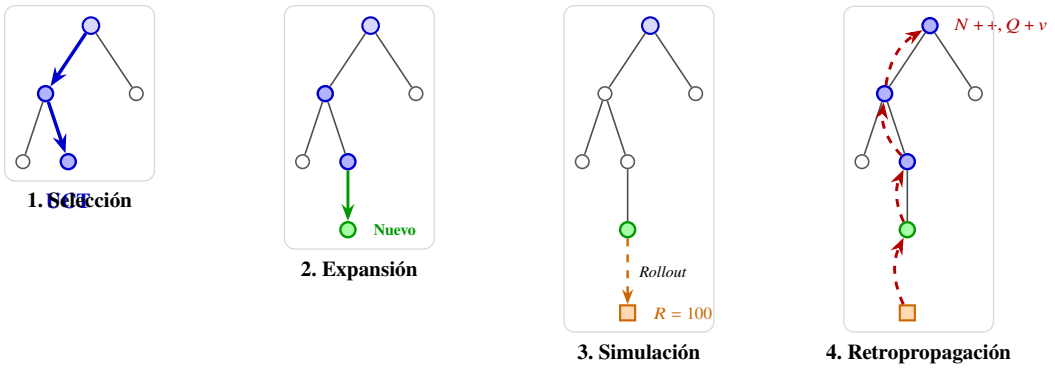


Figura 3.3: Ciclo iterativo de cuatro fases en Monte Carlo Tree Search (MCTS): Selección de nodos mediante la cota UCT, Expansión de nuevas acciones legales, Simulación estocástica (*rollout*) hasta un estado terminal y Retropropagación de recompensas hacia la raíz.

cómputo en acciones de baja calidad. El algoritmo UCT (*Upper Confidence Bounds applied to Trees*) trata este dilema entre explotación y exploración de manera formal [19], [38].

Definición 3.11: Criterio de selección UCT

En el marco de Monte Carlo Tree Search [19], [37], para cada nodo interno v del árbol de búsqueda, la política de selección **UCT** (*Upper Confidence Bounds applied to Trees*) [38], fundamentada en la cota UCB1 para problemas de bandidos multi-brazo [39], elige descender hacia el nodo hijo v^* que maximiza la cota superior de confianza:

$$v^* = \arg \max_{v' \in \text{hijos}(v)} \left[\frac{Q(v')}{N(v')} + C \cdot \sqrt{\frac{\ln N(v)}{N(v')}} \right] \tag{3.33}$$

donde $Q(v')$ denota la recompensa acumulada del nodo hijo v', $N(v')$ es el número de visitas recibidas por dicho hijo, $N(v)$ es el número total de visitas registradas en el nodo padre v, y $C > 0$ es la constante teórica de exploración (frecuentemente establecida como $C = \sqrt{2}$ para recompensas normalizadas en el intervalo $[0, 1]$).

El primer sumando, $\frac{Q(v')}{N(v')}$, es el componente de explotación: favorece las ramas que han dado una tasa de éxito promedio alta en las simulaciones previas. El segundo, $C \cdot \sqrt{\frac{\ln N(v)}{N(v')}}$, es el término de exploración: mientras el nodo padre se sigue visitando y una rama secundaria v' no se elige, el cociente crece de forma monótona, lo que empuja al árbol a examinar alternativas poco muestreadas y evita que converja demasiado pronto a óptimos locales.

En *General Game Playing*, UCT ha sido clave porque ofrece una estrategia general y asintóticamente consistente para dirigir la búsqueda en árboles de juego, sin necesitar funciones de evaluación diseñadas a mano para cada dominio. Goldwaser y Thielscher destacan que los agentes más competitivos de GGP han basado su desempeño en esta combinación de búsqueda por simulación y cotas de confianza, que además sirvió de base para desarrollos recientes con redes neuronales profundas [28].

3.14. Aprendizaje por refuerzo y aprendizaje profundo

El aprendizaje por refuerzo parte de una escena sencilla: un agente en un entorno mira el estado, actúa, recibe una recompensa y cae en otro estado. No hay un maestro que diga en cada paso qué era lo correcto, como ocurre en el aprendizaje supervisado; en un juego, la señal de que algo salió bien o mal puede llegar solo al terminar la partida, y el agente tiene que averiguar qué jugadas la provocaron.

En los juegos, el resultado final (ganar, perder o empatar) puede servir para ajustar la política de decisión del agente. Al combinar esto con redes neuronales profundas se habla de aprendizaje por refuerzo profundo, capaz de aproximar funciones de valor o de política muy complejas, aunque a cambio pide muchos datos, un entrenamiento intensivo y una buena forma de representar el estado.

Con aprendizaje profundo el problema se complica en GGP. No basta con que el agente aprenda a jugar bien: tiene que hacerlo con juegos que le llegan descritos desde fuera, y eso obliga a que la representación del estado y de las acciones sirva para cualquiera de ellos. Por esa razón lo tratamos aquí como antecedente y como línea de investigación avanzada, y no como algo que este proyecto vaya a implementar.

3.15. AlphaZero como antecedente moderno

AlphaZero es un antecedente importante porque combinó autojuego, aprendizaje por refuerzo profundo y búsqueda Monte Carlo para alcanzar un alto desempeño en Ajedrez, Shogi y Go [7]. Su arquitectura usa una red neuronal que estima una política de acciones y un valor de estado, mientras que MCTS afina la elección de movimientos con búsqueda.

Sin embargo, AlphaZero no resuelve por sí mismo el problema central de este Trabajo Terminal. Aunque usa un algoritmo general para más de un juego, necesita simuladores y representaciones adecuadas para cada dominio. En un GGP el reto es otro: interpretar reglas externas, construir el modelo del juego y lograr que el motor de decisión funcione sin conocimiento específico codificado para cada caso.

Goldwaser y Thielscher llevan ideas de aprendizaje por refuerzo profundo al contexto de GGP y muestran que existe una línea de investigación que intenta unir descripciones formales de juegos con métodos inspirados en AlphaZero [28]. Para el alcance de este trabajo, esas ideas son una referencia del estado del arte y ayudan a acotar el proyecto: se busca una arquitectura general e interpretable, no replicar un sistema de entrenamiento profundo a gran escala.

3.16. Síntesis del marco teórico

Desarrollar un General Game Player exige integrar varias áreas de la inteligencia artificial y de las ciencias computacionales. Primero, una representación formal del juego que separe estados, acciones, transiciones, terminalidad y recompensas. Segundo, un lenguaje de descripción o DSL que exprese las reglas sin codificarlas directamente en el motor. Tercero, un intérprete que convierta esa descripción en una representación interna consultable. Y por último, un agente que elija acciones con algoritmos generales de búsqueda o simulación.

Esa integración se organiza en un flujo desacoplado, que muestra la Figura 3.4. El analizador léxico y sintáctico procesa el archivo de reglas externo y crea en memoria el modelo formal Γ. Sobre este modelo, el generador de estados resuelve consultas abstractas de legalidad, transiciones y condiciones terminales, y con eso el motor de decisión (Minimax, poda alfa-beta o MCTS) elige la mejor acción disponible $\mathbf{a}^*$ sin guardar conocimiento particular del dominio.

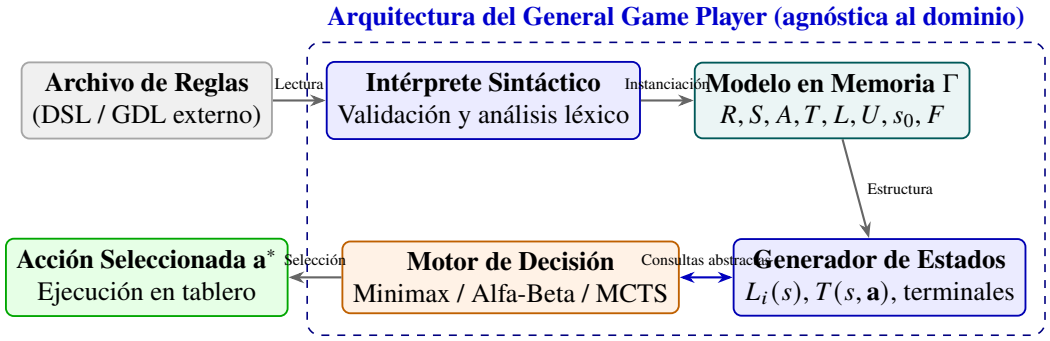


Figura 3.4: Flujo conceptual de ejecución y arquitectura desacoplada del General Game Player.

Lo que aporta este trabajo no es el mejor jugador de un juego en particular, sino una arquitectura desacoplada: el motor debe seguir igual cuando cambia la descripción del juego. Por eso el Gato sirve como caso funcional completo para validar el flujo de reglas, estados y terminalidad, y el Ajedrez como caso controlado de mayor complejidad. Con esta delimitación se puede estudiar la generalidad operativa sin confundir el prototipo con un motor especializado de competencia.

Capítulo 4

Análisis

Capítulo 5

Diseño

Capítulo 6

Desarrollo

Capítulo 7

Pruebas

Capítulo 8

Conclusiones y trabajo a futuro

Referencias

[1] S. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, en, 3.ª ed. Prentice Hall, 2010.

[2] A. L. Samuel, «Some Studies in Machine Learning Using the Game of Checkers,» en, *IBM Journal of Research and Development*, vol. 3, n.º 3, págs. 210-229, 1959. DOI: 10.1147/rd.33.0210.

[3] A. M. Turing, «Computing Machinery and Intelligence,» en, *Mind*, vol. 59, n.º 236, págs. 433-460, 1950. DOI: 10.1093/mind/LIX.236.433.

[4] M. Campbell, A. J. Hoane y F.-h. Hsu, «Deep Blue,» en, *Artificial Intelligence*, vol. 134, n.º 1–2, págs. 57-83, 2002. DOI: 10.1016/S0004-3702(01)00129-1.

[5] J. Schaeffer, N. Burch, Y. Björnsson, A. Kishimoto, M. Müller, R. Lake, P. Lu y S. Sutphen, «Checkers Is Solved,» en, *Science*, vol. 317, n.º 5844, págs. 1518-1522, 2007. DOI: 10.1126/science.1144079.

[6] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel y D. Hassabis, «Mastering the Game of Go with Deep Neural Networks and Tree Search,» en, *Nature*, vol. 529, n.º 7587, págs. 484-489, 2016. DOI: 10.1038/nature16961.

[7] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan y D. Hassabis, «A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go Through Self-Play,» en, *Science*, vol. 362, n.º 6419, págs. 1140-1144, 2018. DOI: 10.1126/science.aar6404.

[8] M. Genesereth, N. Love y B. Pell, «General Game Playing: Overview of the AAAI Competition,» en, *AI Magazine*, vol. 26, n.º 2, págs. 62-72, 2005. dirección: https://logic.stanford.edu/publications/genesereth/aaai.pdf.

[9] M. Genesereth y M. Thielscher, *General Game Playing*, en. Morgan & Claypool Publishers, 2014. dirección: https://logic.stanford.edu/ggp/chapters/cover.html.

[10] N. Love, T. Hinrichs, D. Haley, E. Schkufza y M. Genesereth, «General Game Playing: Game Description Language Specification,» en, Stanford Logic Group, inf. téc. LG-2006-01, 2008. dirección: https://logic.stanford.edu/ggp/.

[11] C. E. Shannon, «Programming a Computer for Playing Chess,» en, *Philosophical Magazine*, vol. 41, n.º 314, págs. 256-275, 1950. DOI: 10.1080/14786445008521796.

[12] S. Chinchalkar, «An Upper Bound for the Number of Reachable Positions,» en, *ICCA Journal*, vol. 19, n.º 3, págs. 181-183, 1996. DOI: 10.3233/ICG-1996-19309.

[13] D. Levy y M. Newborn, *How Computers Play Chess*, en. New York: Computer Science Press, 1991.

[14] M. Schmid, M. Moravcik, N. Burch, R. Kadlec, J. Davidson, K. Waugh, N. Bard, F. Timbers, M. Lanctot, Z. Holland, T. El-Assal y M. Bowling, «Student of Games: A unified learning algorithm for both perfect and imperfect information games,» en, *Science Advances*, vol. 9, n.º 46, eadg3256, 2023. DOI: 10.1126/sciadv.adg3256.

[15] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap y D. Silver, «Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model,» en, *Nature*, vol. 588, n.º 7839, págs. 604-609, 2020. DOI: 10.1038/s41586-020-03051-4.

[16] É. Piette, D. J. N. J. Soemers, M. Stephenson, C. F. Sironi, M. H. M. Winands y C. Browne, «Ludii – The General Game System,» en, en *Proceedings of the 2020 IEEE Conference on Games (CoG)*, 2020, págs. 438-445. DOI: 10.1109/CoG47356.2020.9231548.

[17] J. Kowalski, M. Mika, J. Sutowicz y M. Szykuła, «Regular Boardgames,» en, en *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence*, 2019, págs. 1699-1706. DOI: 10.1609/aaai.v33i01.33011699.

[18] H. Finnsson, «Generalized Monte-Carlo Tree Search Extensions for General Game Playing,» en, en *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*, 2012, págs. 1550-1556.

[19] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis y S. Colton, «A Survey of Monte Carlo Tree Search Methods,» en, *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, n.º 1, págs. 1-43, 2012. DOI: 10.1109/TCIAIG.2012.2186810.

[20] J. Clune, «Heuristic Evaluation Methods for General Game Playing,» en, en *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence*, 2007, págs. 1133-1139.

[21] S. Schiffel y M. Thielscher, «Fluxplayer: A Successful General Game Player,» en, en *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence*, 2007, págs. 1191-1196.

[22] H. Finnsson e Y. Björnsson, «Simulation-Based Approach to General Game Playing,» en, en *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*, 2008, págs. 259-264.

[23] S. Draper y A. Rose, *Sancho: An Industrial-Strength General Game Playing Agent*, en, Stanford Logic Group General Game Playing Competition, International GGP Competition Winner, 2014.

[24] S. Gelly y D. Silver, «Monte-Carlo tree search and rapid action value estimation in computer Go,» en, *Artificial Intelligence*, vol. 175, n.º 11, págs. 1856-1875, 2011. DOI: 10.1016/j.artint.2011.03.007.

[25] M. Thielscher, «A General Game Description Language for Incomplete Information and Stochastic Games,» en, en *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*, 2010, págs. 994-999. DOI: 10.1609/aaai.v24i1.7677.

[26] D. J. N. J. Soemers, E. Piette, M. Stephenson y C. Browne, «The Ludii Game Description Language is Universal,» en, en *Proceedings of the 2024 IEEE Conference on Games (CoG)*, 2024, págs. 1-8. DOI: 10.1109/CoG60054.2024.10645550.

[27] M. Lanctot, E. Lockhart, J.-B. Lespiau, V. Zambaldi, S. Upadhyay, J. Pérolat, S. Srinivasan, F. Timbers, K. Tuyls, S. Omidshafiei, D. Hennes, D. Morrill, P. Feng, T. Biro, J. Hou, F. Bazzana, M. Schmid, D. Geiger, S. Liu, J. Schrittwieser, I. Antonoglou, T. Graepel y M. Bowling, «OpenSpiel: A Framework for Reinforcement Learning in Games,» en, *arXiv preprint arXiv:1908.09453*, 2019. DOI: 10.48550/arXiv.1908.09453.

[28] A. Goldwaser y M. Thielscher, «Deep Reinforcement Learning for General Game Playing,» en, en *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence*, 2020, págs. 1701-1708. DOI: 10.1609/aaai.v34i02.5533.

[29] H. W. Kuhn, «Extensive Games and the Problem of Information,» en, en *Contributions to the Theory of Games (AM-28), Volume II*, Princeton University Press, 1953, págs. 193-216. DOI: 10.1515/9781400881970-012.

[30] M. J. Osborne y A. Rubinstein, *A Course in Game Theory*, en. Cambridge, MA: MIT Press, 1994.

[31] J. McCarthy y P. J. Hayes, «Some Philosophical Problems from the Standpoint of Artificial Intelligence,» en, en *Machine Intelligence 4*, B. Meltzer y D. Michie, eds., Edinburgh University Press, 1969, págs. 463-502.

[32] R. Reiter, «On Closed World Data Bases,» en, en *Logic and Data Bases*, H. Gallaire y J. Minker, eds., Plenum Press, 1978, págs. 55-76. DOI: 10.1007/978-1-4684-3384-5_3.

[33] B. Bonet y H. Geffner, «Planning as Heuristic Search,» en, *Artificial Intelligence*, vol. 129, n.º 1–2, págs. 5-33, 2001. DOI: 10.1016/S0004-3702(01)00108-4.

[34] A. Cropper, R. Evans y M. Law, «Inductive General Game Playing,» en, *Machine Learning*, vol. 109, págs. 1393-1434, 2020. DOI: 10.1007/s10994-019-05843-w.

[35] J. von Neumann, «Zur Theorie der Gesellschaftsspiele,» de, *Mathematische Annalen*, vol. 100, n.º 1, págs. 295-320, 1928. DOI: 10.1007/BF01448847.

[36] D. E. Knuth y R. W. Moore, «An Analysis of Alpha-Beta Pruning,» en, *Artificial Intelligence*, vol. 6, n.º 4, págs. 293-326, 1975. DOI: 10.1016/0004-3702(75)90019-3.

[37] R. Coulom, «Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search,» en, en *Computers and Games (CG 2006)*, ép. Lecture Notes in Computer Science, vol. 4630, Springer, 2007, págs. 72-83. DOI: 10.1007/978-3-540-75538-8_7.

[38] L. Kocsis y C. Szepesvári, «Bandit Based Monte-Carlo Planning,» en, en *Machine Learning: ECML 2006*, ép. Lecture Notes in Computer Science, vol. 4212, Springer, 2006, págs. 282-293. DOI: 10.1007/11871842_29.

[39] P. Auer, N. Cesa-Bianchi y P. Fischer, «Finite-time Analysis of the Multiarmed Bandit Problem,» en, *Machine Learning*, vol. 47, n.º 2–3, págs. 235-256, 2002. DOI: 10.1023/A:1013689704352.

Anexos

8.1. Cronogramas